



第5章 运输层

- ◆ 端口和套接字

- ◆ TCP协议

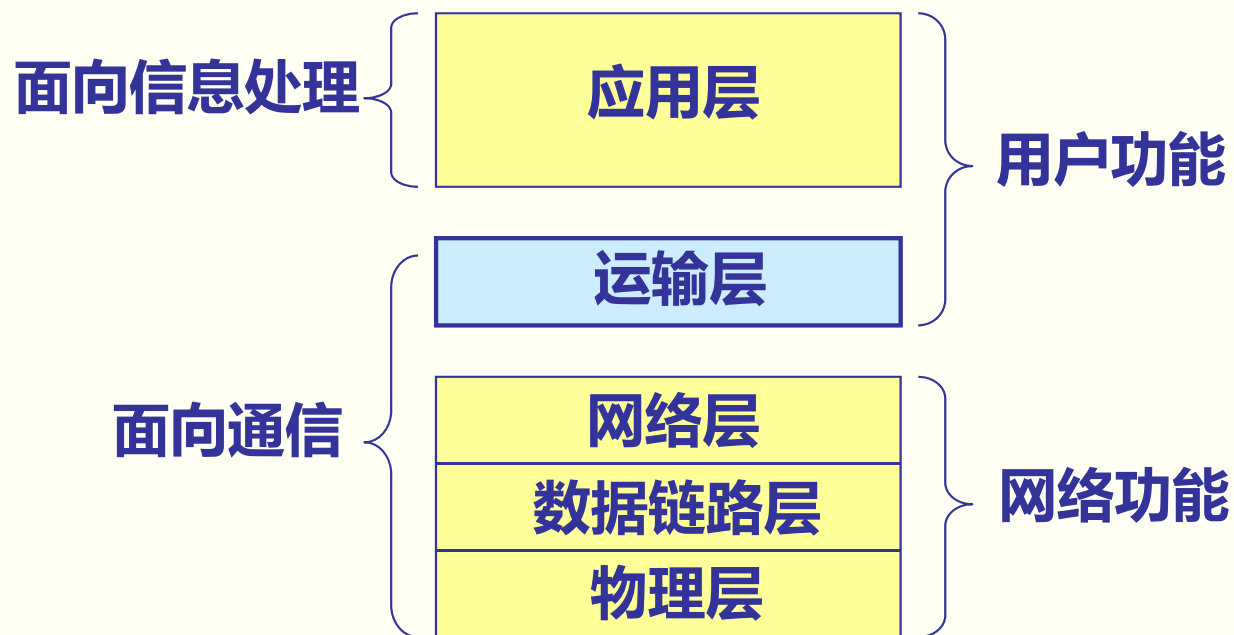
- ◆ UDP协议



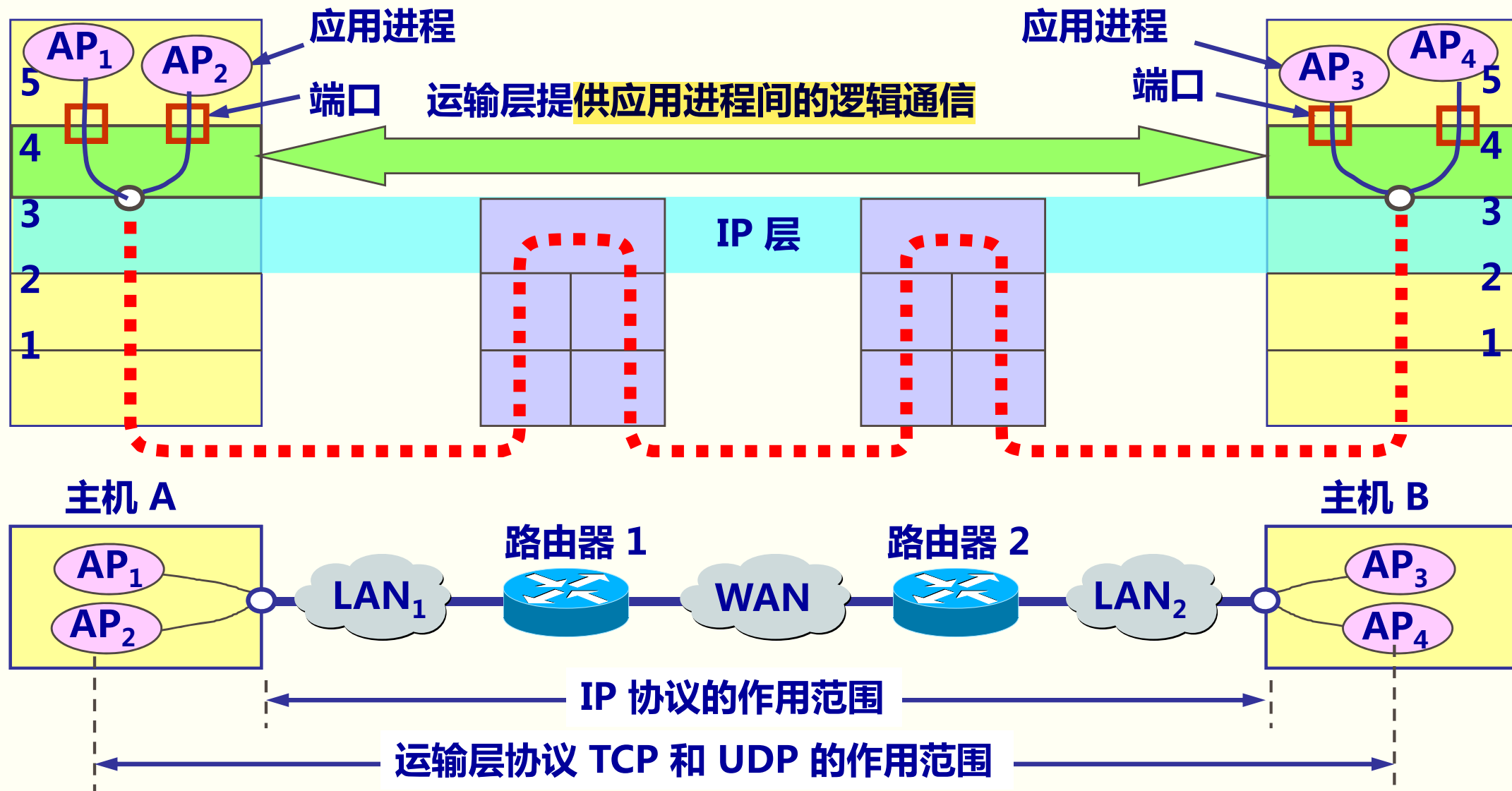
5.1 运输层协议概述

5.1.1 进程之间的通信

从通信和信息处理的角度看，运输层向它上面的应用层提供通信服务，它属于面向通信部分的最高层，同时也是用户功能中的最低层。



运输层的作用*



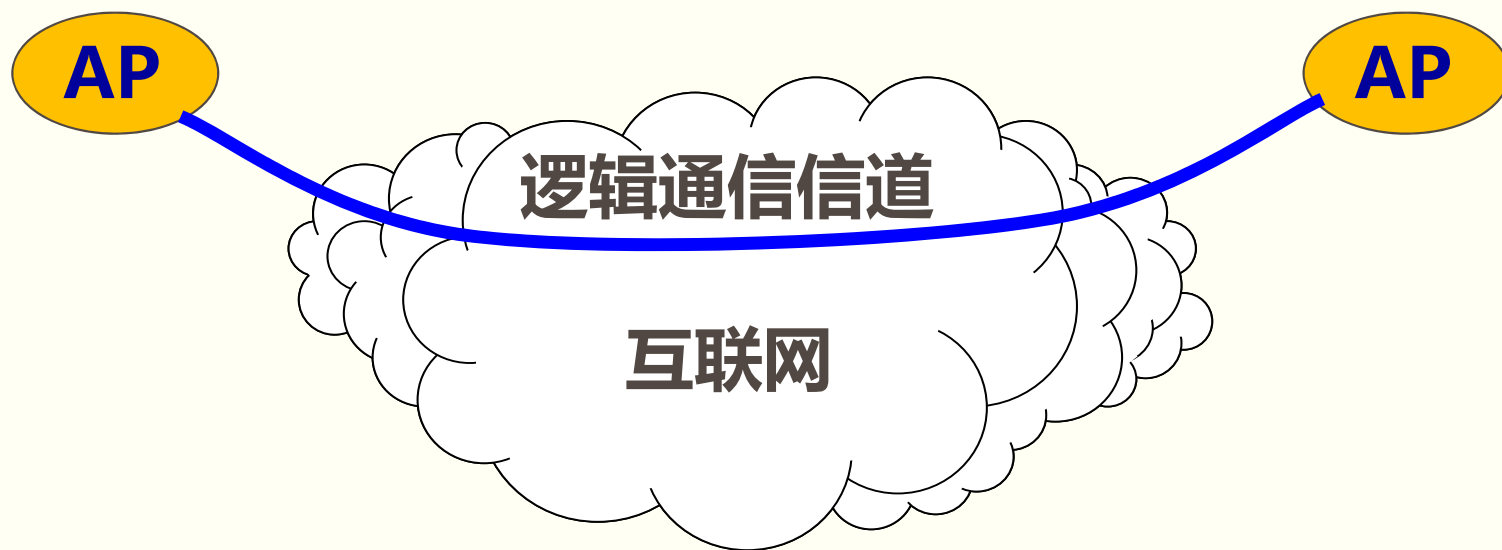
运输层为相互通信的应用进程提供了逻辑通信

运输层的作用

- ◆两个主机进行通信实际上就是两个主机中的应用进程互相通信。
- ◆应用进程之间的通信又称为端到端的通信。
- ◆运输层的一个很重要的功能就是复用和分用。应用层不同进程的报文通过不同的端口向下交到运输层，再往下就共用网络层提供的服务。
- ◆“运输层提供应用进程间的逻辑通信”。“逻辑通信”的意思是：运输层之间的通信好像是沿水平方向传送数据。但事实上这两个运输层之间并没有一条水平方向的物理连接。

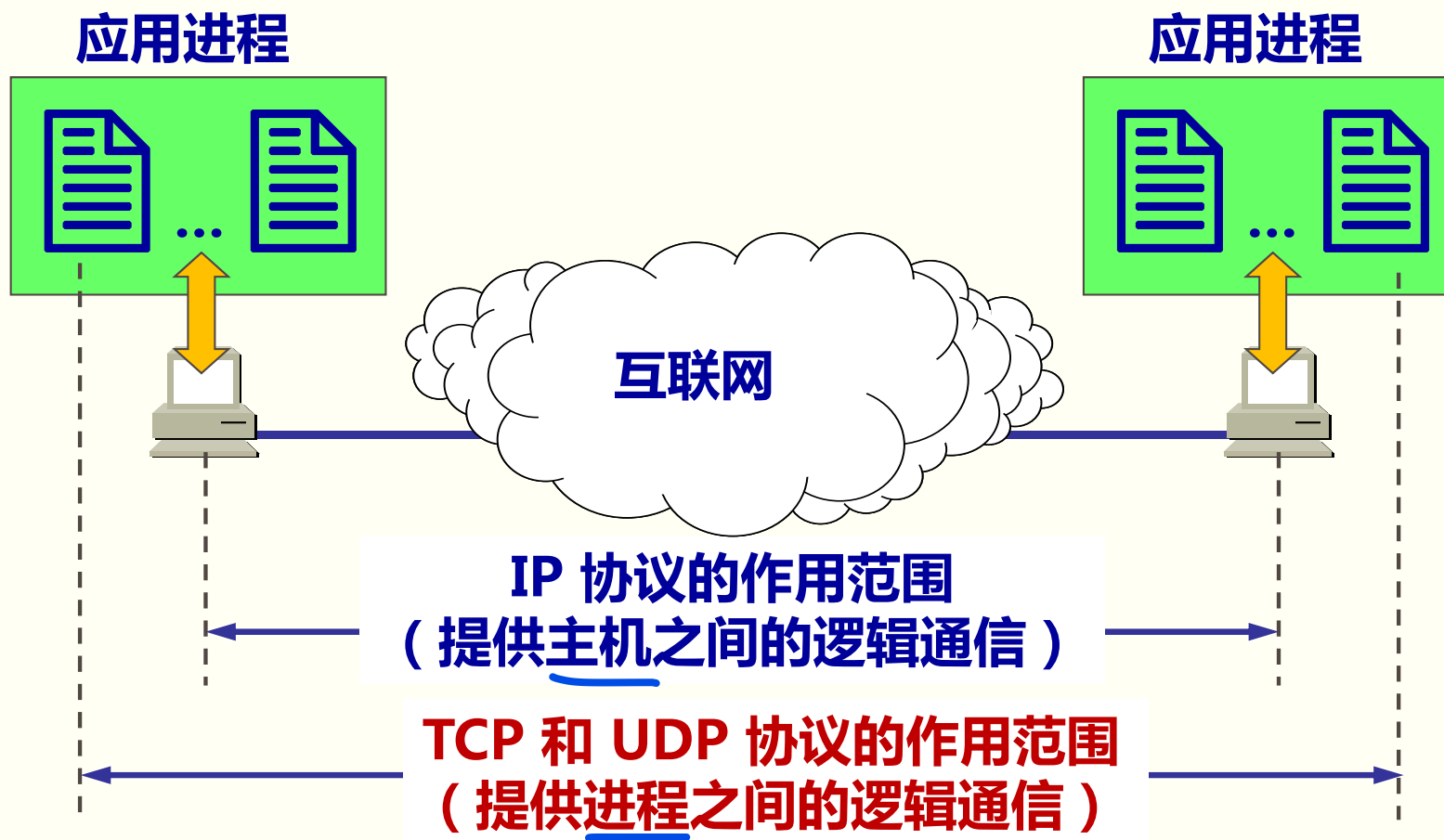
屏蔽作用

◆运输层向高层用户**屏蔽**了下面网络核心的细节（如网络拓扑、所采用的路由选择协议等），它使应用进程看见的就是好像在两个运输层实体之间有一条**端到端的逻辑通信信道**。



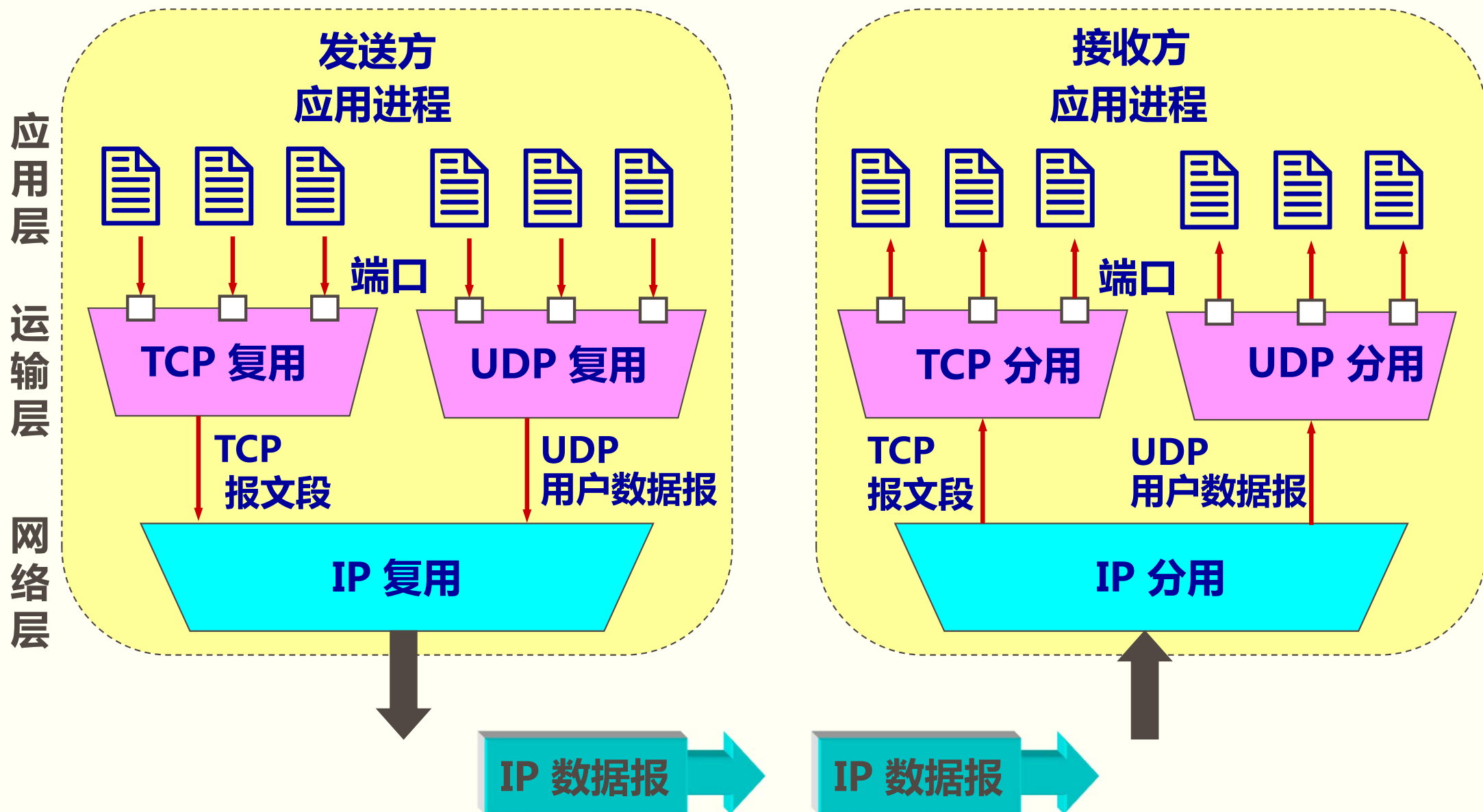
网络层协议和运输层协议的主要区别

网络层是为主机之间提供逻辑通信，
而运输层为应用进程之间提供端到端的逻辑通信。



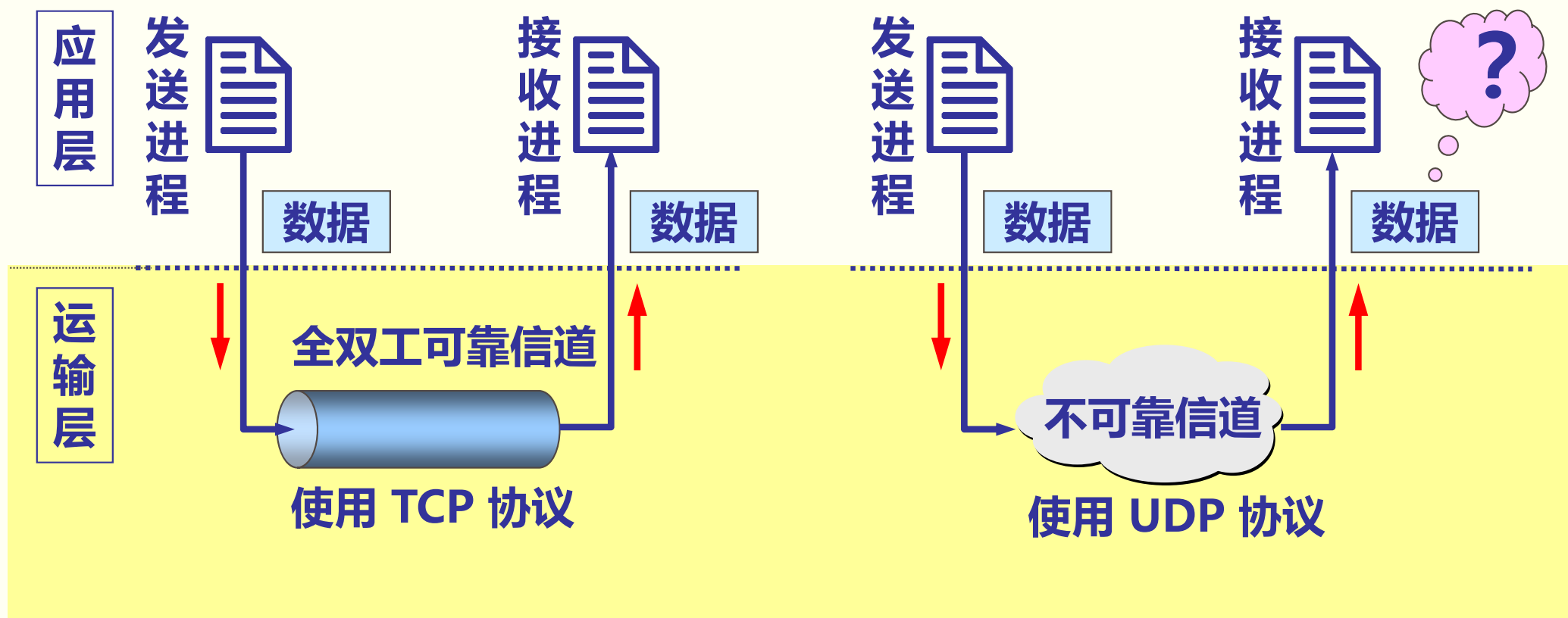
运输层协议和网络层协议的主要区别

基于端口的复用和分用功能



两种不同的运输协议

- ◆当运输层采用**面向连接的 TCP** 协议时，尽管下面的网络是**不可靠的**（只提供尽最大努力服务），但这种逻辑通信信道就相当于一**条全双工的可靠信道**。
- ◆当运输层采用**无连接的 UDP** 协议时，这种逻辑通信信道是一条**不可靠信道**。



5.1.2 运输层的两个主要协议***

TCP/IP 的运输层有两个主要协议：

(1) **用户数据报协议 UDP** (User Datagram Protocol)：

传送的数据单位协议是 **UDP 报文或用户数据报**

(2) **传输控制协议 TCP** (Transmission Control Protocol)：

传送的数据单位协议是 **TCP 报文段**(segment)



TCP/IP 体系中的运输层协议

TCP 与 UDP的特点***

◆UDP：一种无连接协议

- 提供无连接服务。在传送数据之前不需要先建立连接。
- 传送的数据单位协议是 UDP 报文或用户数据报。
- 对方的运输层在收到 UDP 报文后，不需要给出任何确认。
- 虽然 UDP 不提供可靠交付，但在某些情况下 UDP 是一种最有效的工作方式。

◆TCP：一种面向连接的协议

- ◆提供面向连接的服务。
- ◆传送的数据单位协议是 TCP 报文段 (segment)。
- ◆TCP 不提供广播或多播服务。
- ◆由于 TCP 要提供可靠的、面向连接的运输服务，因此不可避免地增加了许多的开销。这不仅使协议数据单元的首部增大很多，还要占用许多的处理机资源。

还要强调两点

- ◆ 运输层的 UDP 用户数据报与网际层的IP数据报有很大区别。
 - IP 数据报要经过互连网中许多路由器的存储转发。
 - UDP 用户数据报是在运输层的端到端抽象的逻辑信道中传送的。
- ◆ TCP 报文段是在运输层抽象的端到端逻辑信道中传送，这种信道是可靠的全双工信道。但这样的信道却不知道究竟经过了哪些路由器，而这些路由器也根本不知道上面的运输层是否建立了 TCP 连接。

5.1.3 运输层的端口*

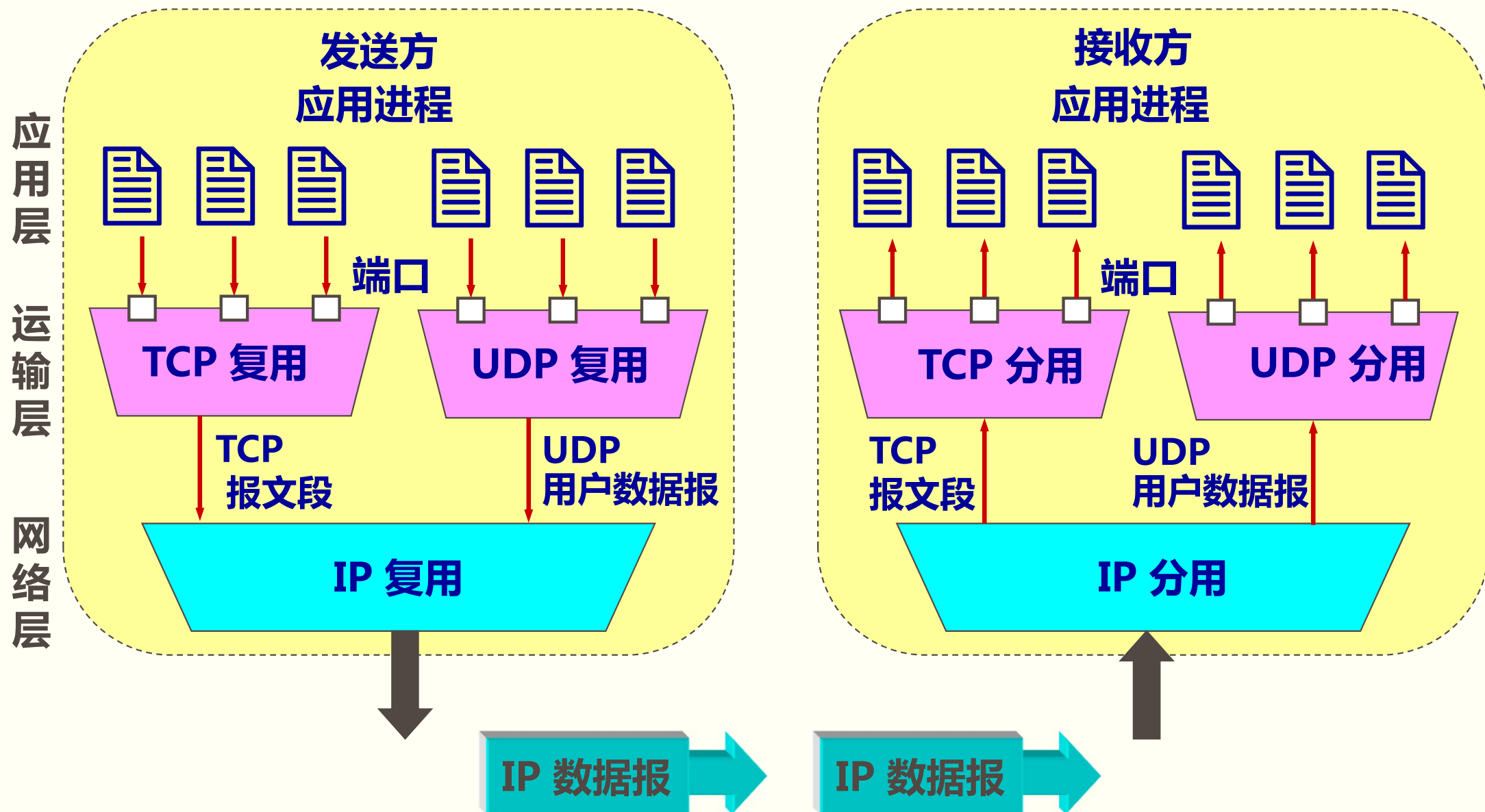
◆运行在计算机中的进程是用进程标识符来标志的。为了使运行不同操作系统的计算机的应用进程能够互相通信，就必须用统一的方法对 TCP/IP 体系的应用进程进行标志。

◆端口就是运输层服务访问点。

◆端口的作用就是让应用层的各种应用进程都能将其数据通过端口向下交付给运输层，以及让运输层知道应当将其报文段中的数据向上通过端口交付给应用层相应的进程。端口是用来标志应用层的进程。

端口：让数据在应用层与运输层之间交付

端口（port）在进程之间的通信中所起的作用



软件端口与硬件端口**

- ◆两个不同的概念。
- ◆在协议栈层间的抽象的协议端口是软件端口。
- ◆路由器或交换机上的端口是硬件端口。
- ◆硬件端口是不同硬件设备进行交互的接口，而软件端口是应用层的各种协议进程与运输实体进行层间交互的一种地址。

运输层端口***

- ◆端口用一个 16 位端口号进行标志。
- ◆端口号只具有本地意义，即端口号只是为了标志本计算机应用层中的各进程。
- ◆在互联网中，不同计算机的相同端口号是没有联系的。

由此可见，两个计算机中的进程要互相通信，不仅必须知道对方的 IP 地址（为了找到对方的计算机），而且还要知道对方的端口号（为了找到对方计算机中的应用进程）。

两大类端口**

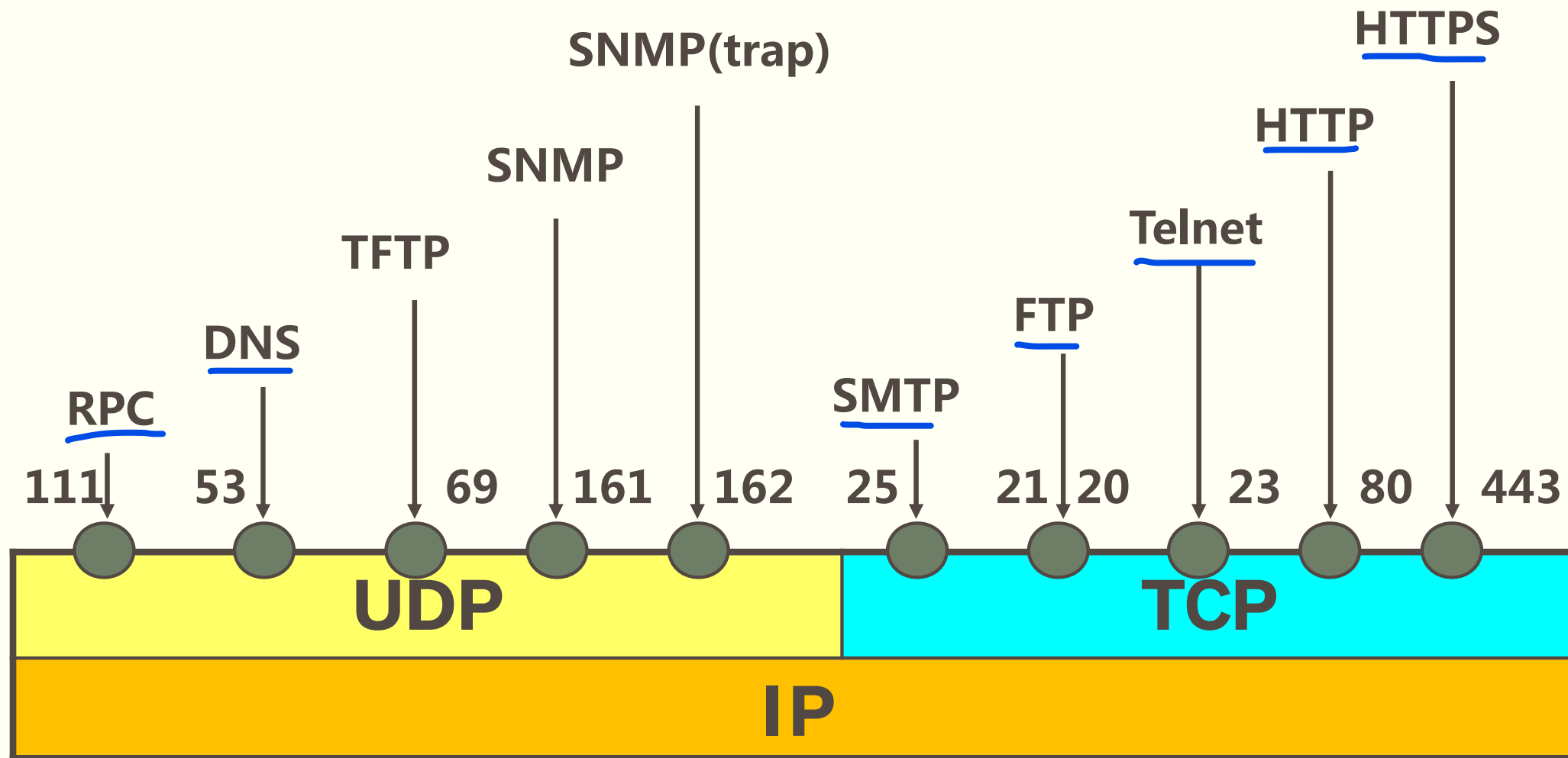
(1) 服务器端使用的端口号

- **熟知端口**，数值一般为 0~1023。
- **登记端口号**，数值为 1024~49151，为没有熟知端口号的应用程序使用的。
使用这个范围的端口号**必须在 IANA 登记**，以防止重复。

(2) 客户端使用的端口号

- 又称为**短暂端口号**，数值为 49152~65535，**留给客户进程选择暂时使用。**
- 当服务器进程**收到客户进程的报文时**，就知道了客户进程**所使用的动态端口号**。
通信结束后，这个端口号**可供其他客户进程以后使用。**

常用的熟知端口





5.2 用户数据报协议UDP

5.2.1 UDP概述***

◆UDP 在 IP 的数据报服务之上增加的功能：复用和分用、差错检测

◆UDP 的主要特点：

(1) UDP 是无连接的，发送数据之前不需要建立连接，因此减少了开销和发送数据之前的时延。

(2) UDP 使用尽最大努力交付，即不保证可靠交付，因此主机不需要维持复杂的连接状态表。

(3) UDP 是面向报文的。UDP 对应用层交下来的报文，既不开并，也不拆分，而是保留这些报文的边界。UDP 一次交付一个完整的报文。

(4) UDP 没有拥塞控制，因此网络出现的拥塞不会使源主机的发送速率降低。这对某些实时应用是很重要的。很适合多媒体通信的要求。

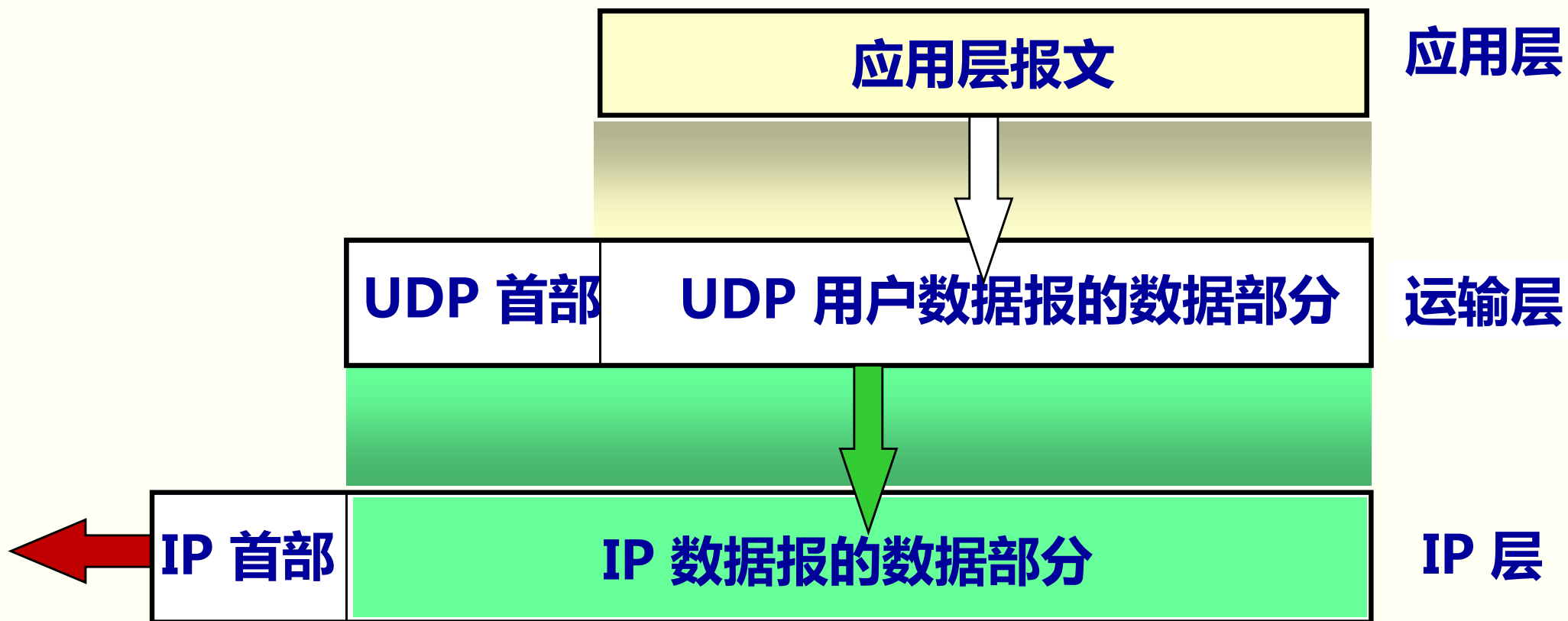
(5) UDP 支持一对一、一对多、多对一和多对多的交互通信。

(6) UDP 的首部开销小，只有 8 个字节，比 TCP 的 20 个字节的首部要短。

面向报文的 UDP

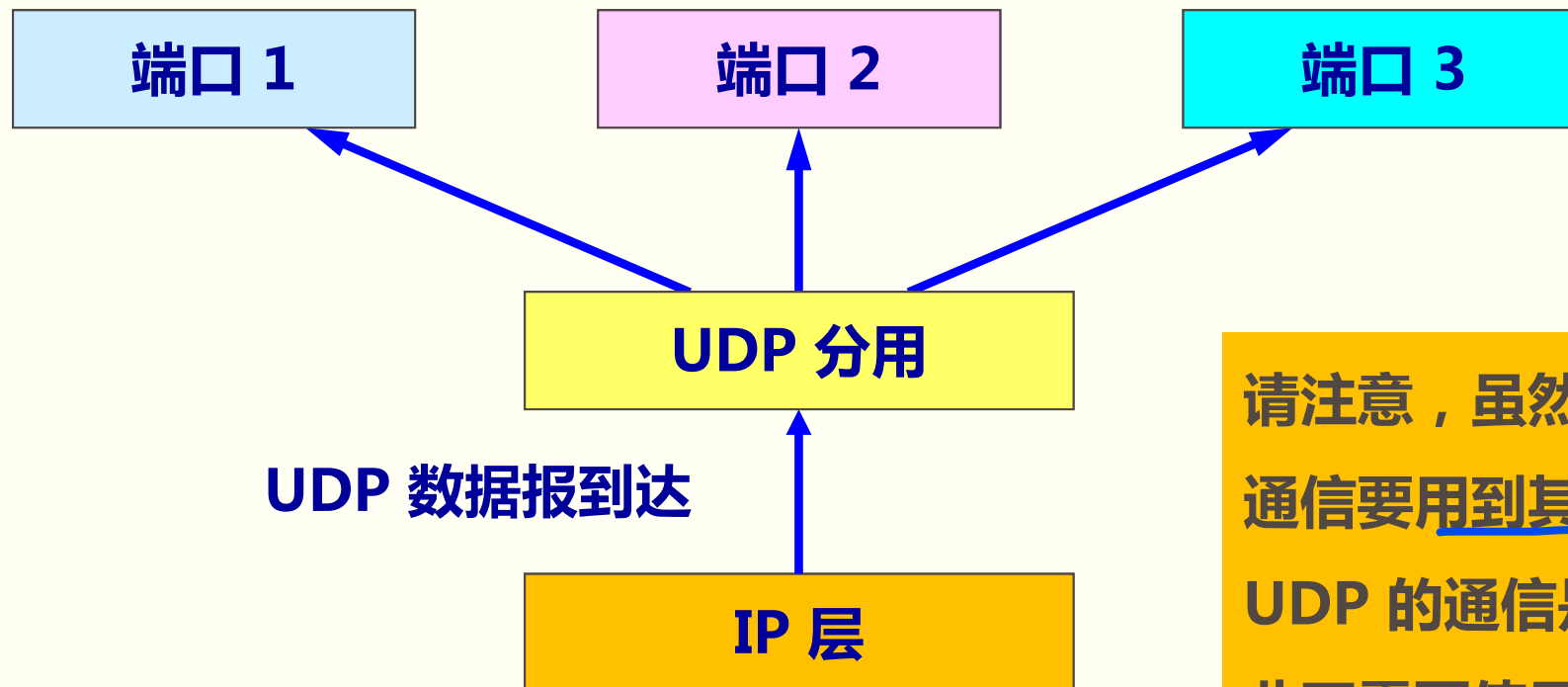
- ◆发送方 UDP 对应用程序交下来的报文，在**添加首部后就向下交付 IP 层**。UDP 对应用层交下来的报文，**既不合并，也不拆分**，而是保留这些报文的边界。应用层交给 UDP 多长的报文，UDP 就照样发送，即**一次发送一个报文**。
- ◆接收方 UDP 对 IP 层交上来的 UDP 用户数据报，在**去除首部后就原封不动地交付上层的应用进程**，**一次交付一个完整的报文**。
- ◆应用程序必须**选择合适大小的报文**。
 - 若**报文太长**，UDP 把它交给 IP 层后，IP 层在传送时可能要进行**分片**，这会降低 IP 层的**效率**。
 - 若**报文太短**，UDP 把它交给 IP 层后，会使 IP 数据报的**首部的相对长度太大**，这也降低了 IP 层的**效率**。

UDP 是面向报文的



UDP 基于端口的分用

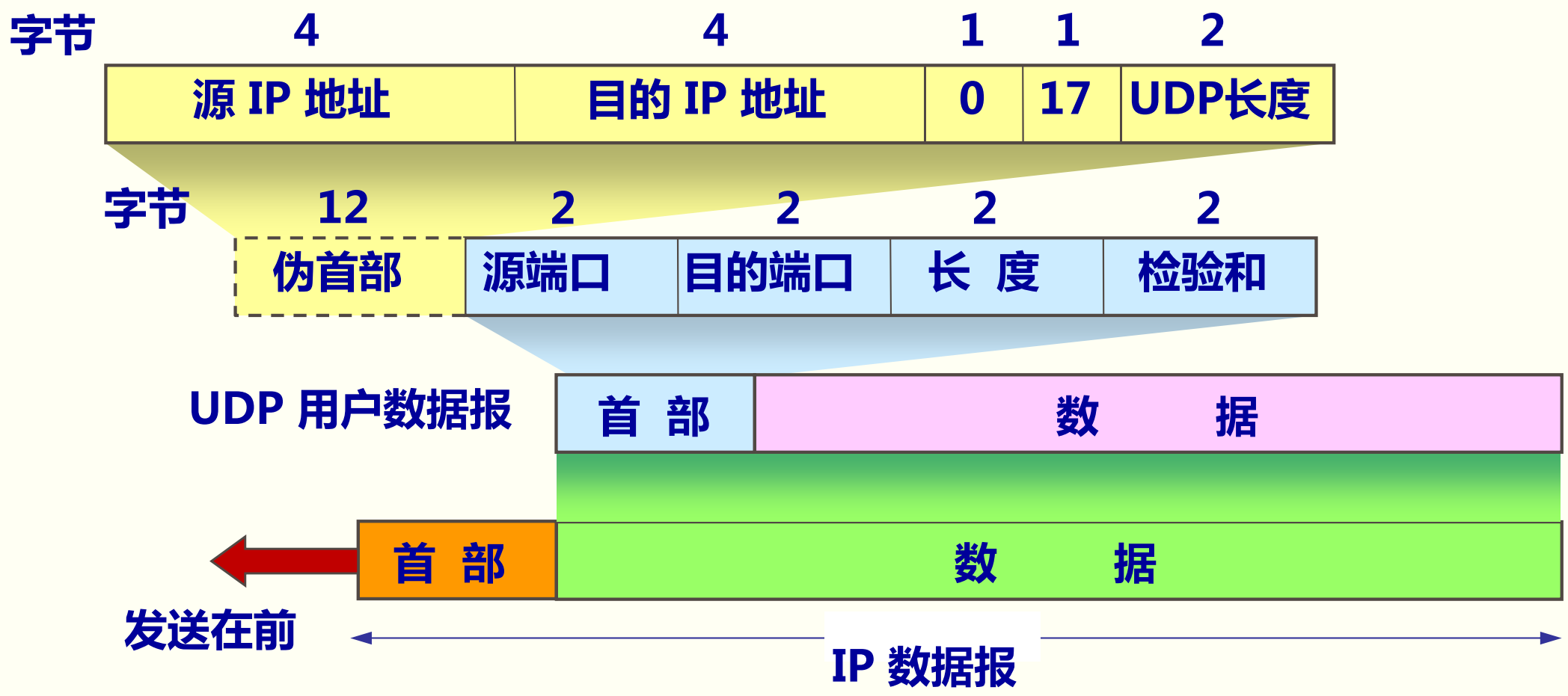
当运输层从 IP 层 收到 UDP 数据报 时，就根据首部中的目的端口，把 UDP 数据报 通过相应的端口，上交最后的终点——应用进程。



请注意，虽然在 UDP 之间的通信要用到其端口号，但由于 UDP 的通信是无连接的，因此不需要使用套接字。

5.2.2 UDP 的首部格式**

用户数据报 UDP 有两个字段：数据字段和首部字段。首部字段有 8 个字节。在计算检验和时，临时把“伪首部”和 UDP 用户数据报连接在一起。伪首部仅仅是为了计算检验和。



计算 UDP 校验和的例子

12 字节 伪首部	153.19.8.104			
	171.3.14.11			
8 字节 UDP 首部	全 0	17	15	
	1087		13	
7 字节 数据	15		全 0	
	数据	数据	数据	数据
	数据	数据	数据	全 0

填充

UDP的校验和是把首部
和数据部分一起都
检验。

按二进制反码运算求和
将得出的结果求反码

10011001	00010011	→	153.19
00001000	01101000	→	8.104
10101011	00000011	→	171.3
00001110	00001011	→	14.11
00000000	00010001	→	0 和 17
00000000	00001111	→	15
00000100	00111111	→	1087
00000000	00001101	→	13
00000000	00001111	→	15
00000000	00000000	→	0 (校验和)
01010100	01000101	→	数据
01010011	01010100	→	数据
01001001	01001110	→	数据
01000111	00000000	→	数据和 0 (填充)
10010110	11101101	→	求和得出的结果
01101001	00010010	→	校验和



5.3 传输控制协议 TCP 概述

5.3.1 TCP 最主要的特点***

◆TCP 是**面向连接**的运输层协议。

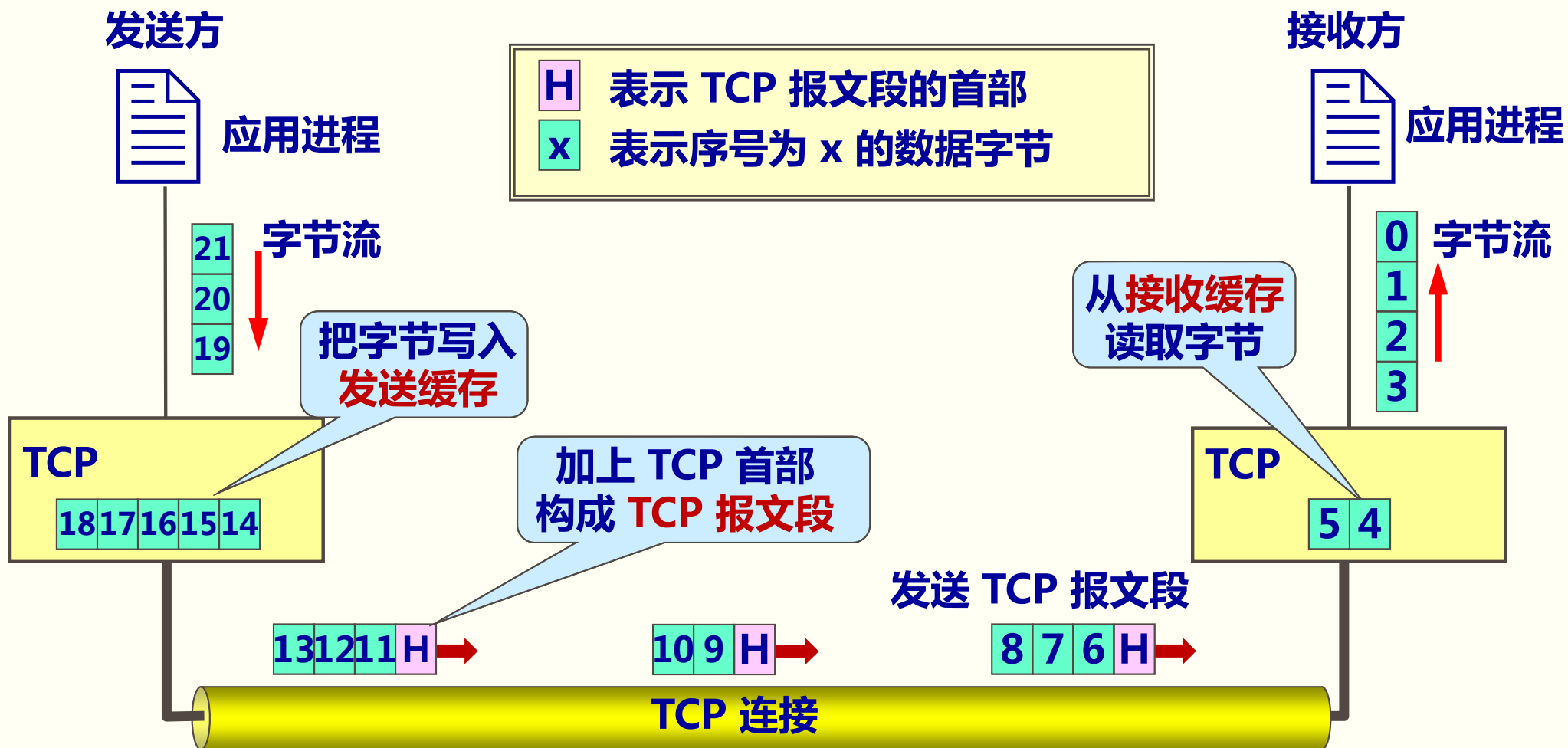
◆每一条 TCP 连接**只能有两个端点** (endpoint) , 每一条 TCP 连接**只能是对点的** (一对一) 。

◆TCP 提供**可靠交付**的服务。

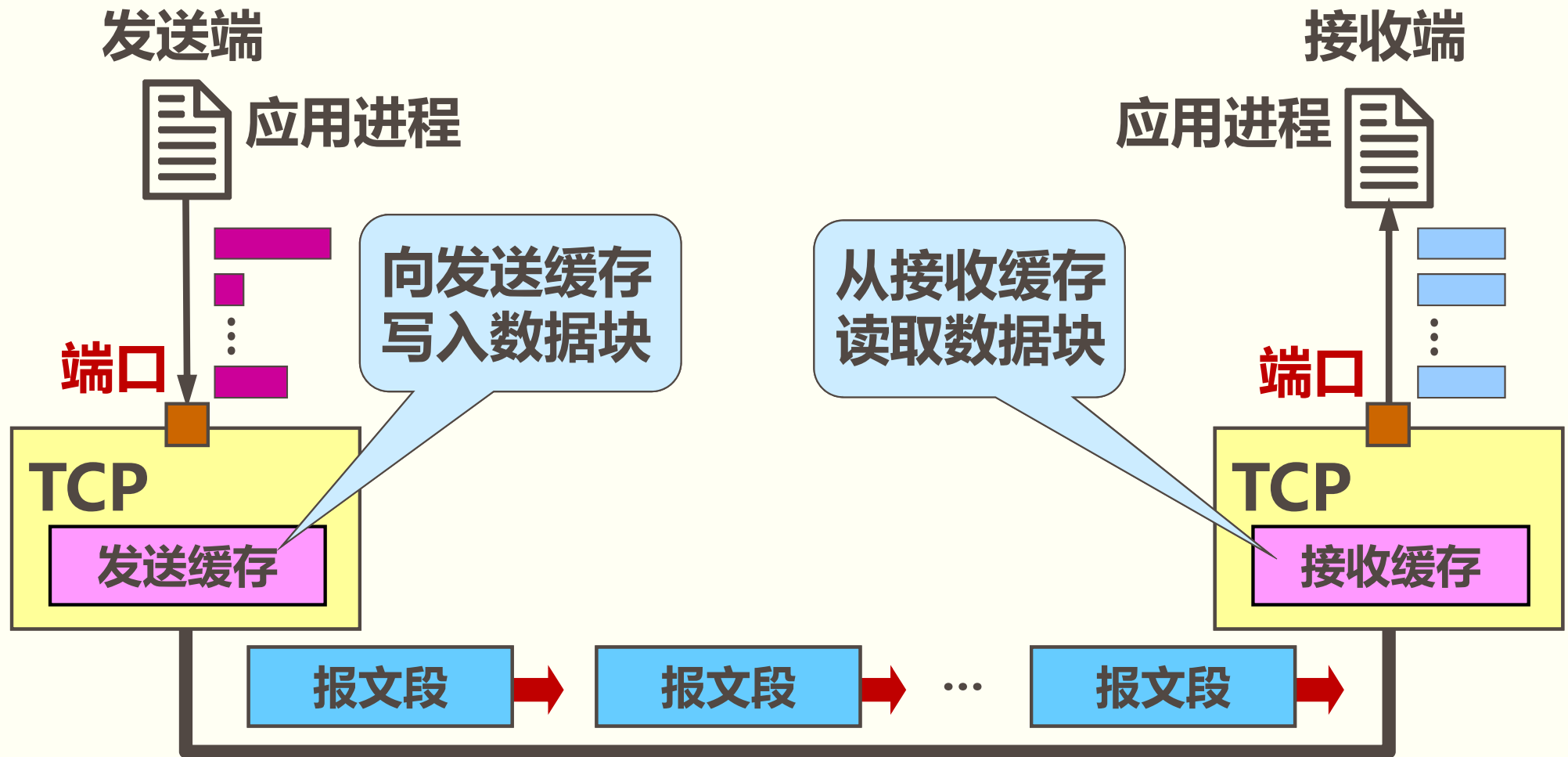
◆TCP 提供**全双工通信**。

◆**面向字节流** : TCP 中的 “流” (stream)指的是**流入或流出进程的字节序列**。 “面向字节流” 的含义是 : 虽然应用程序和 TCP 的交互是**一次一个数据块** , 但 TCP 把应用程序交下来的数据看成仅仅是一**连串无结构的字节流**。 TCP **不保证**接收方和发送方的**数据块具有对应大小的关系**。但接收方和发送方应用程序发出的**字节流必须完全一样**。

TCP 面向流的概念



TCP 面向流的概念



- TCP **不关心**应用进程一次把多长的报文发送到 TCP 缓存。
- TCP 对连续的字节流进行分段，形成 TCP 报文段。

注意

- ◆TCP 连接是一条虚连接而不是一条真正的物理连接。
- ◆TCP 对应用进程一次把多长的报文发送到TCP 的缓存中是不关心的
- ◆TCP 根据对方给出的窗口值和当前网络拥塞的程度来决定一个报文段应包含多少个字节（UDP 发送的报文长度是应用进程给出的）。
- ◆TCP 可把太长的数据块划分短一些再传送。
- ◆TCP 也可等待积累有足够多的字节后再构成报文段发送出去。

5.3.2 TCP 的连接**

- ◆TCP 把连接作为**最基本的抽象**。
- ◆每一条 TCP 连接**有两个端点**。
- ◆TCP **连接的端点**不是主机，不是主机的IP 地址，不是应用进程，也不是运输层的协议端口。**TCP 连接的端点叫做套接字 (socket) 或插口。**
- ◆**端口号拼接到 (concatenated with) IP 地址即构成了套接字。**

套接字 socket = (IP地址 : 端口号)

- ◆每一条 TCP 连接**唯一**地被通信两端的**两个端点**（即两个套接字）所确定。

即：

TCP 连接 ::= {socket1, socket2} = {(IP1: port1) , (IP2: port2)}

TCP 连接，IP 地址，套接字

◆TCP 连接就是由协议软件所提供的一种抽象。TCP 连接的端点是个很抽象的套接字，即（IP 地址：端口号）。同一个 IP 地址可以有多个不同的 TCP 连接。同一个端口号也可以出现在多个不同的 TCP 连接中。

◆Socket 的其他含义：

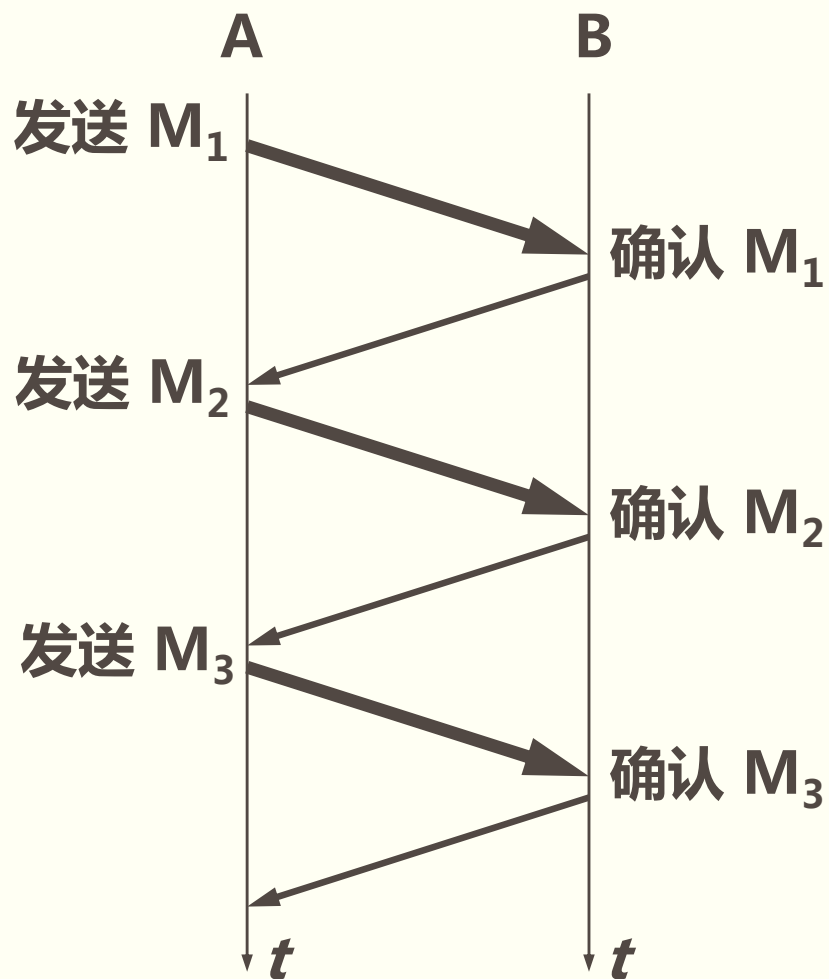
- 应用编程接口 API 称为 socket API, 简称为 socket。
- socket API 中使用的一个函数名也叫作 socket。
- 调用 socket 函数的端点称为 socket。
- 调用 socket 函数时其返回值称为 socket 描述符，可简称为 socket。
- 在操作系统内核中连网协议的 Berkeley 实现，称为 socket 实现。



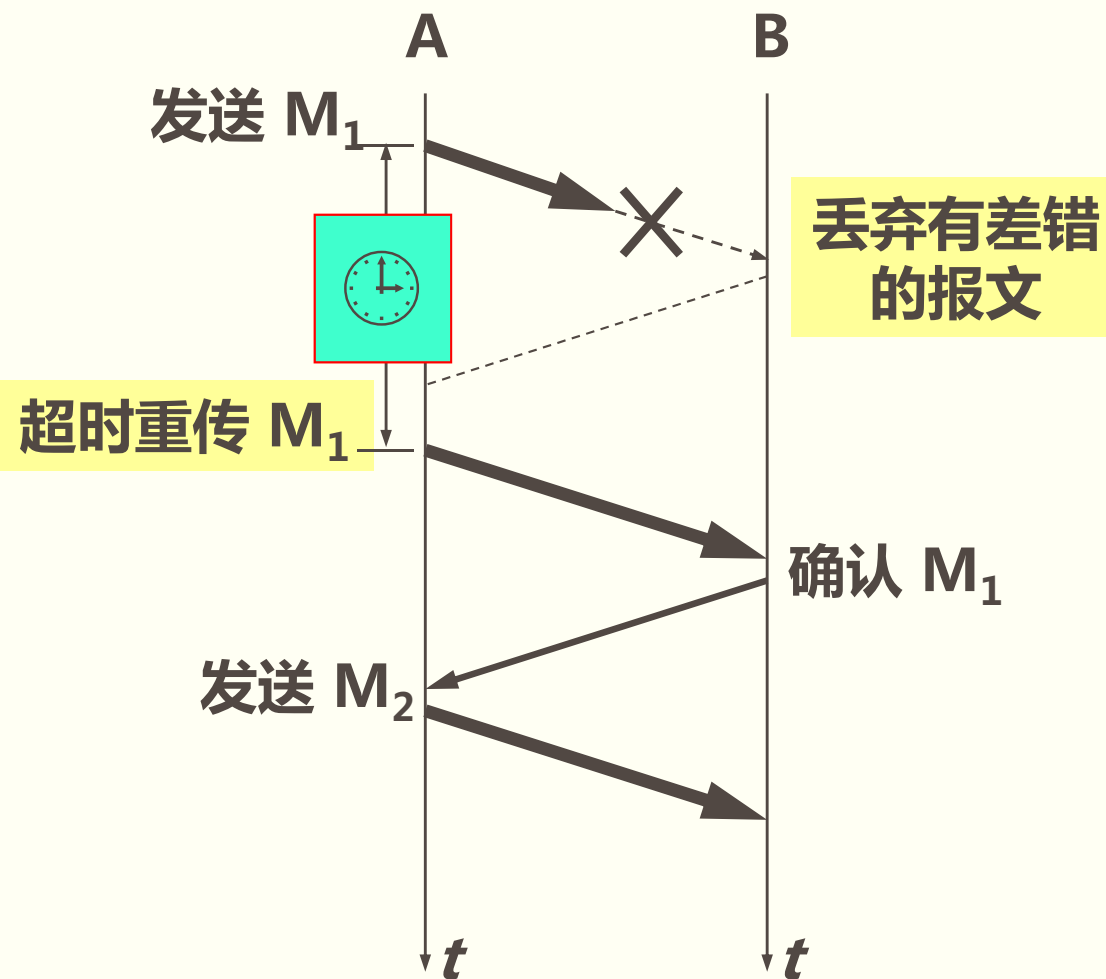
5.4 可靠传输的工作原理*

5.4.1 停止等待协议*

“停止等待”就是每发送完一个分组就停止发送，等待对方的确认。在收到确认后再发送下一个分组。

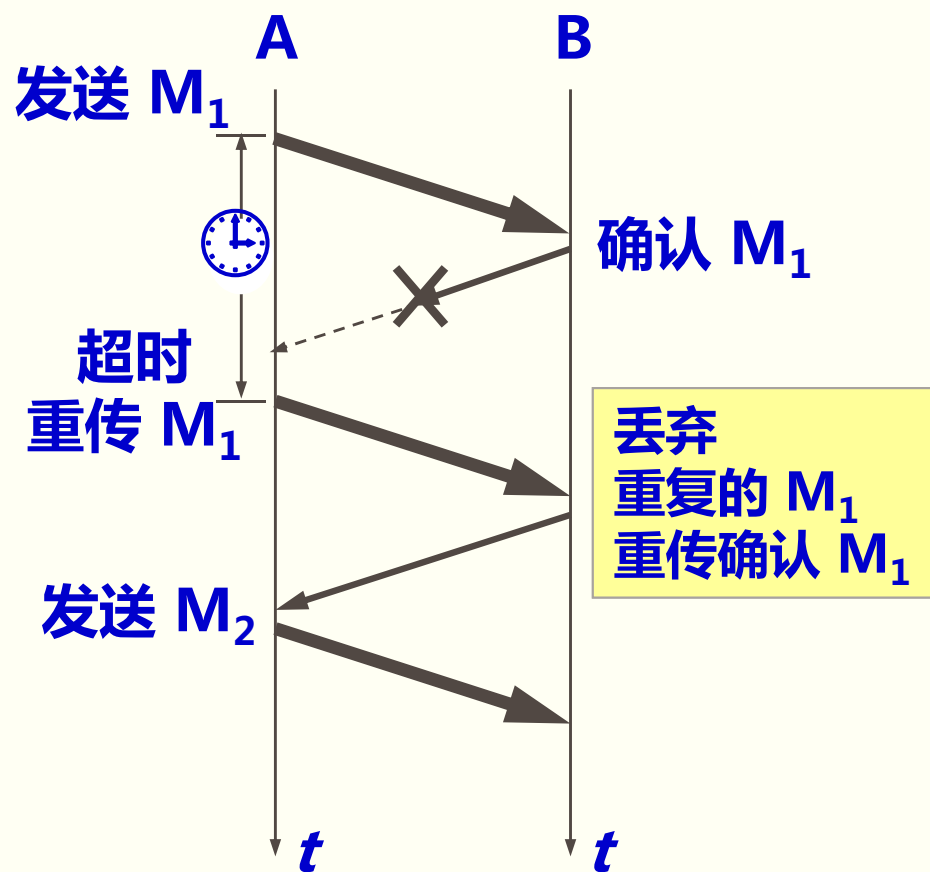


(a) 无差错情况

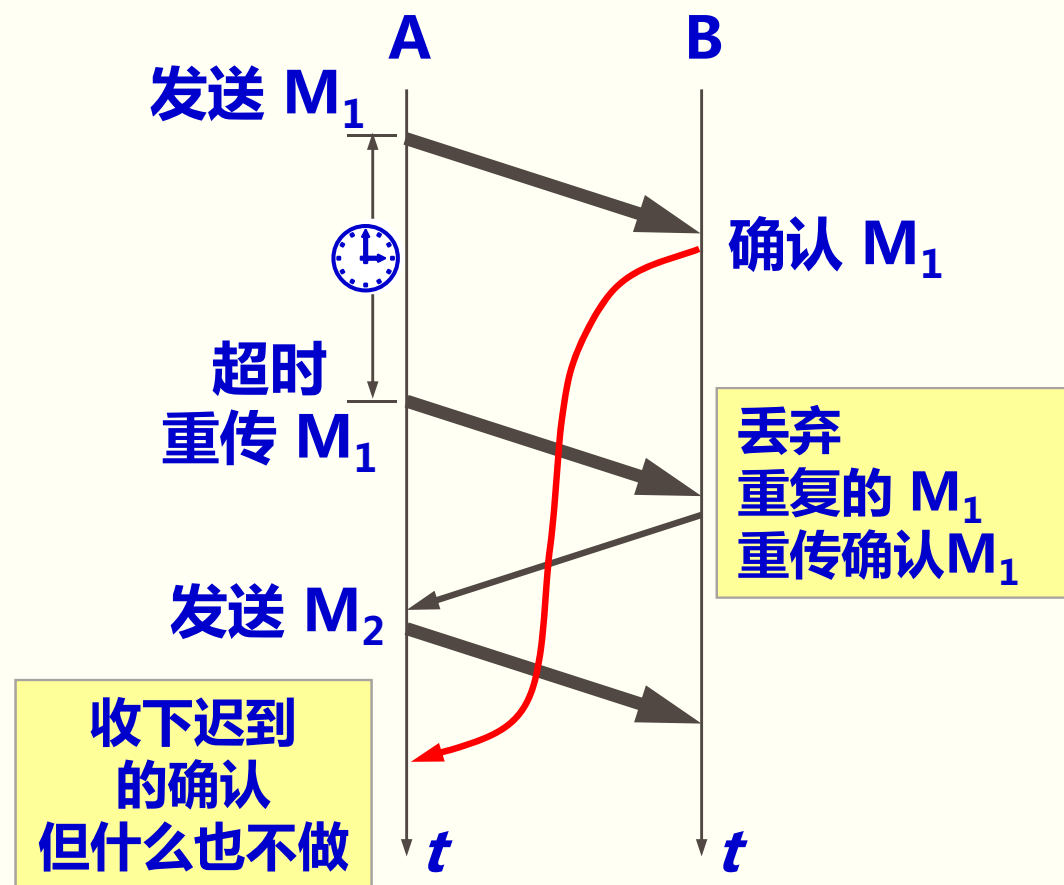


(b) 超时重传：分组错误和分组丢失

3. 确认丢失和确认迟到



(a) 确认丢失



(b) 确认迟到

在发送完一个分组后，必须**暂时保留**已发送的分组的副本，以备**重发**。**分组和确认分组都必须进行编号**。超时计时器的**重传时间**应当比数据在分组传输的**平均往返时间更长一些**。

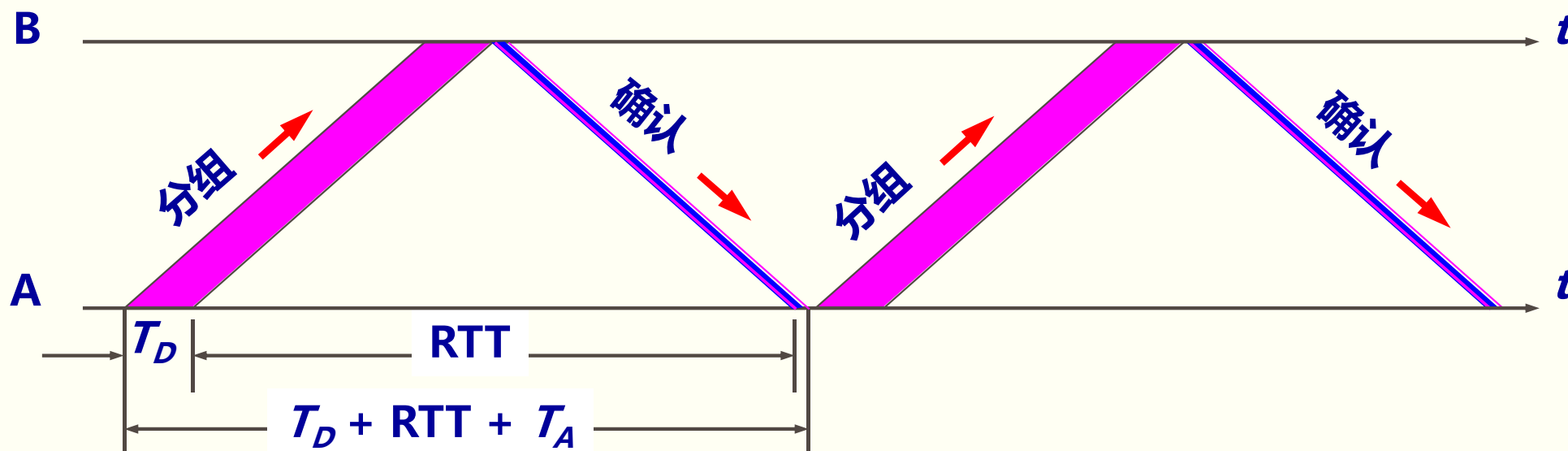
自动重传请求 ARQ**

- ◆通常 A 最终总是可以收到对所有发出的分组的确认。如果 A 不断重传分组但总是收不到确认，就说明通信线路太差，不能进行通信。
- ◆使用上述的确认和重传机制，我们就可以在不可靠的传输网络上实现可靠的通信。
- ◆像上述的这种可靠传输协议常称为自动重传请求 ARQ (Automatic Repeat reQuest)。意思是重传的请求是自动进行的，接收方不需要请求发送方重传某个出错的分组。

4. 信道利用率

停止等待协议的优点是简单，缺点是信道利用率太低。

当往返时间 RTT 远大于分组发送时间 T_D 时，信道的利用率就会非常低。



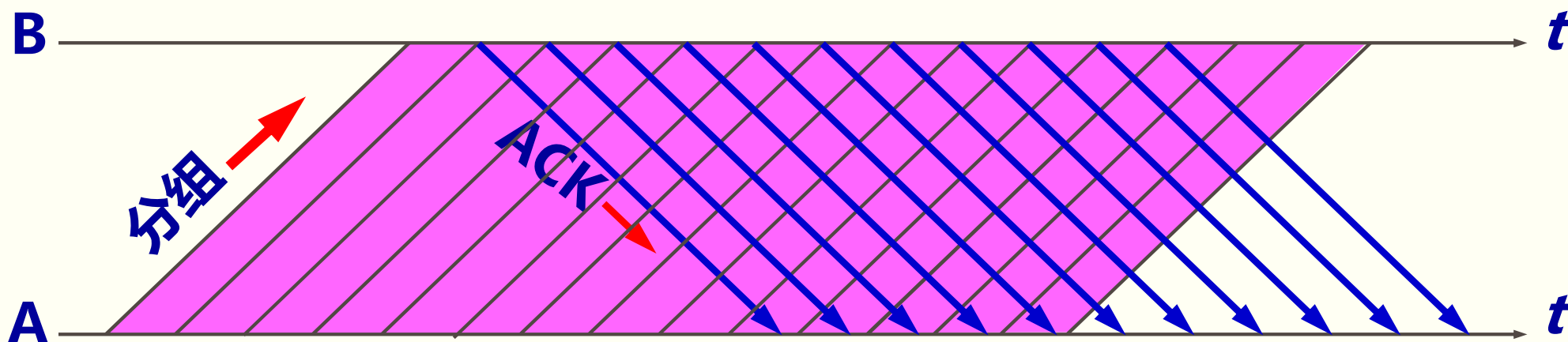
信道利用率

$$U = \frac{T_D}{T_D + RTT + T_A}$$

流水线传输

流水线传输就是发送方可连续发送多个分组，不必每发完一个分组就停顿下来等待对方的确认。这样可使信道上一直有数据不间断地传送。

由于信道上一直有数据不间断地传送，这种传输方式可获得很高的信道利用率。

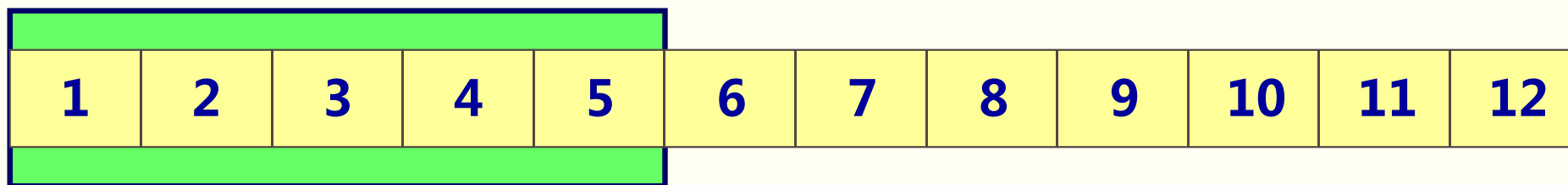


流水线传输可提高信道利用率

5.4.2 连续ARQ协议***

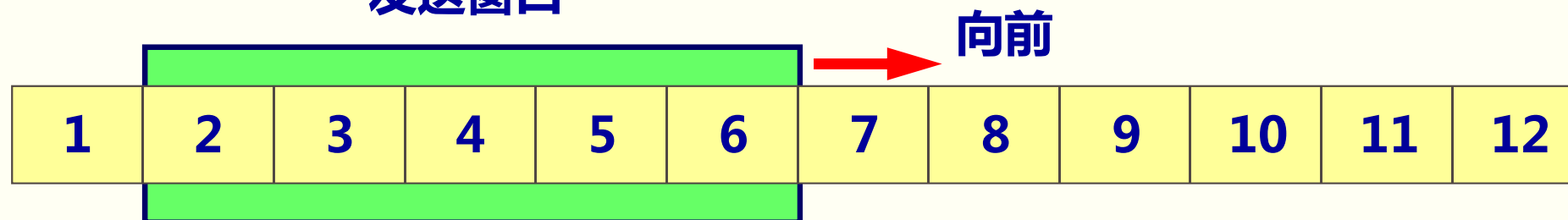
发送方维持的**发送窗口**，它的意义是：**位于发送窗口内的分组都可连续发送出去，而不需要等待对方的确认**。这样，信道利用率就提高了。连续 ARQ 协议规定，**发送方每收到一个确认，就把发送窗口向前滑动一个分组的位置**。

发送窗口



(a) 发送方维持发送窗口（发送窗口是 5）

发送窗口



(b) 收到一个确认后发送窗口向前滑动

累积确认

◆接收方一般采用累积确认的方式。即不必对收到的分组逐个发送确认，而是对按序到达的最后一个分组发送确认，这样就表示：到这个分组为止的所有分组都已正确收到了。

◆优点：容易实现，即使确认丢失也不必重传。

◆缺点：不能向发送方反映出接收方已经正确收到的所有分组的信息。如果发送方发送了前 5 个分组，而中间的第 3 个分组丢失了。这时接收方只能对前两个分组发出确认。发送方无法知道后面三个分组的下落，而只好把后面的三个分组都再重传一次。这就叫做 Go-back-N (回退 N)，表示需要再退回来重传已发送过的 N 个分组。

TCP 可靠通信的具体实现

- ◆TCP 连接的每一端都必须设有两个窗口：一个发送窗口和一个接收窗口。
- ◆TCP 的可靠传输机制用字节的序号进行控制。TCP 所有的确认都是基于序号而不是基于报文段。
- ◆TCP 两端的四个窗口经常处于动态变化之中。
- ◆TCP连接的往返时间 RTT 也不是固定不变的。需要使用特定的算法估算较为合理的重传时间。

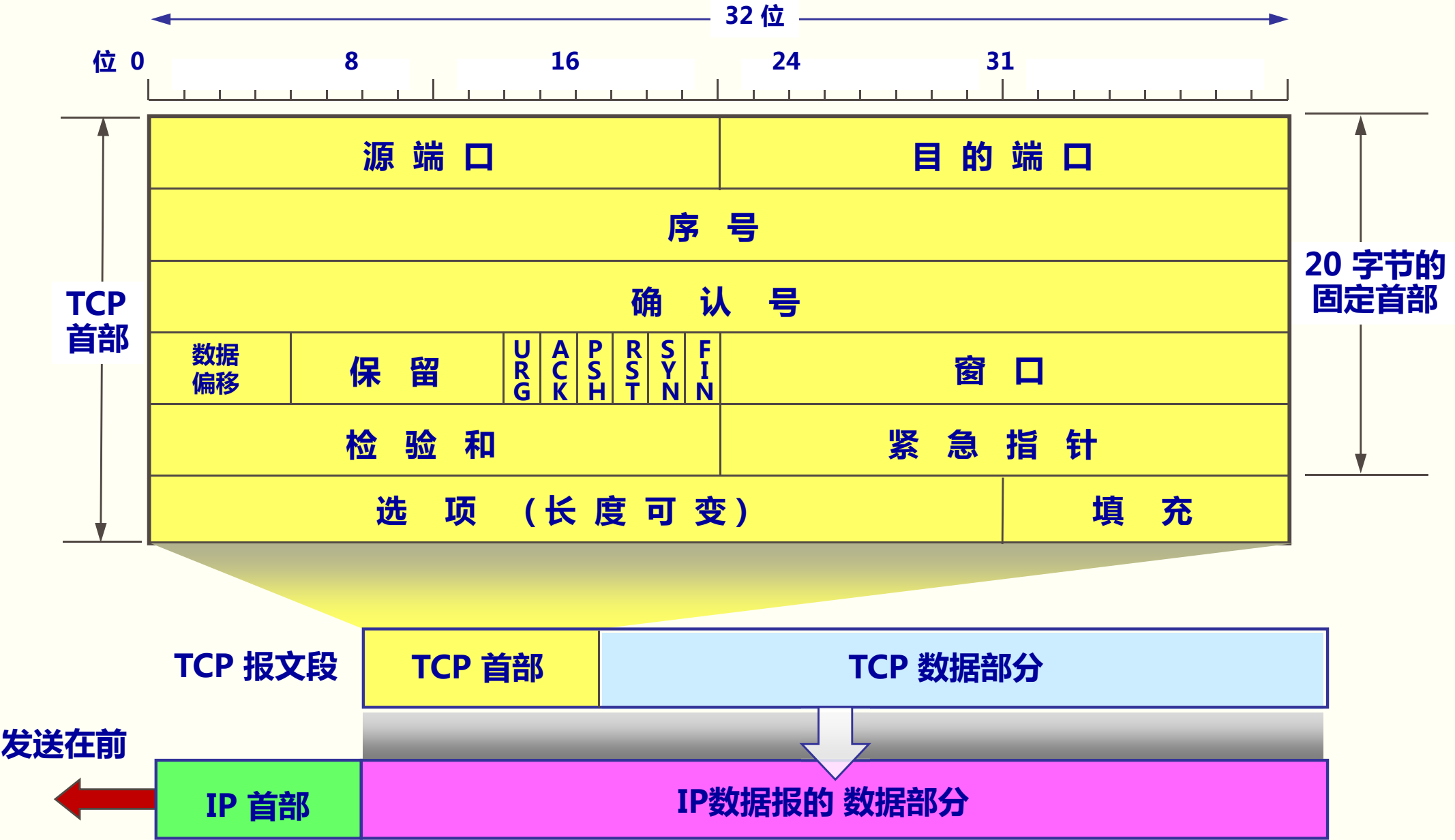


5.5 TCP 报文段的首部格式

5.5 TCP 报文段的首部格式**

- ◆TCP 虽然是面向字节流的，但 TCP 传送的数据单元却是报文段。
- ◆一个 TCP 报文段分为首部和数据两部分，而 TCP 的全部功能都体现在它首部中各字段的作用。
- ◆TCP 报文段首部的前 20 个字节是固定的，后面有 $4n$ 字节是根据需要而增加的选项 (n 是整数)。因此 TCP 首部的最小长度是 20 字节。

TCP 报文段的首部格式



TCP 报文段的首部格式

源 端 口							目 的 端 口						
序 号													
确 认 号													
数据 偏移	保 留			U R G	A C K	P S H	R S T	S Y N	F I N	窗 口			
检 验 和								紧 急 指 针					
选 项 （长 度 可 变）										填 充			

源端口和目的端口字段：各占 2 字节。端口是运输层与应用层的**服务接口**。运输层的复用和分用功能都要通过**端口才能实现**。

序号字段：占 4 字节。TCP 连接中传送的数据流中的**每一个字节都编上一个序号**。序号字段的值则指的是**本报文段所发送的数据的第一个字节的序号**。

确认号字段：占 4 字节，是**期望收到对方的下一个报文段的数据的第一个字节的序号**。

TCP 报文段的首部格式

源 端 口								目 的 端 口							
序 号															
确 认 号															
数据 偏移	保 留		U	A	P	R	S	F	窗 口						
			R	C	S	T	S	N							
检 验 和								紧 急 指 针							
选 项 （长 度 可 变）												填 充			

数据偏移（即首部长度）：占 4 位，它指出 TCP 报文段的**数据起始处距离 TCP 报文段的起始处有多远**。“数据偏移”的单位是 **32 位字**（以 4 字节为计算单位）。

保留字段：占 6 位，**保留**为今后使用，但目前应置为 0。

紧急 URG：当 URG = 1 时，表明**紧急指针字段有效**。它告诉系统此报文段中有**紧急数据**，**应尽快传送(相当于高优先级的数据)**。

确认 ACK：只有当 **ACK = 1** 时确认号字段才有效。当 **ACK = 0** 时，确认号无效。

TCP 报文段的首部格式

源 端 口							目 的 端 口						
序 号													
确 认 号													
数据 偏移	保 留		U R G	A C K	P S H	R S T	S Y N	F I N	窗 口				
检 验 和								紧 急 指 针					
选 项 （长 度 可 变）									填 充				

推送 PSH (PuSH)：接收 TCP 收到 PSH = 1 的报文段，就尽快地交付接收应用进程，而不再等到整个缓存都填满了后再向上交付。

复位 RST (ReSeT)：当 RST = 1 时，表明 TCP 连接中出现严重差错（如由于主机崩溃或其他原因），必须释放连接，然后再重新建立运输连接。

同步 SYN：同步 SYN = 1 表示这是一个连接请求或连接接受报文。

终止 FIN (FINish)：用来释放一个连接。FIN = 1 表明此报文段的发送端的数据已发送完毕，并要求释放运输连接。

TCP 报文段的首部格式

源 端 口								目 的 端 口							
序 号															
确 认 号															
数据 偏移	保 留		U	A	P	R	S	F	窗 口						
			R	C	S	T	S	N							
检 验 和								紧 急 指 针							
选 项 （长 度 可 变）												填 充			

窗口字段：占 2 字节，用来让对方设置发送窗口的依据，单位为字节。

检验和：占 2 字节。检验和字段检验的范围包括首部和数据这两部分。在计算检验和时，要在 TCP 报文段的前面加上 12 字节的伪首部。

紧急指针字段：占 16 位，指出在本报文段中紧急数据共有多少个字节（紧急数据放在本报文段数据的最前面）。

填充字段：这是为了使整个首部长度是 4 字节的整数倍。

为什么要规定 MSS ？

- ◆选项字段：长度可变。TCP 最初只规定了一种选项，即最大报文段长度 MSS。MSS (Maximum Segment Size)是 TCP 报文段中的数据字段的最大长度。数据字段加上 TCP 首部才等于整个的 TCP 报文段。
- ◆若选择较小的 MSS 长度，网络的利用率就降低。当 TCP 报文段只含有 1 字节的数据时，在 IP 层传输的数据报的开销至少有 40 字节(包括 TCP 报文段的首部和 IP 数据报的首部)。这样，对网络的利用率就不会超过 $1/41$ 。到了数据链路层还要加上一些开销。
- ◆若 TCP 报文段非常长，那么在 IP 层传输时就有可能要分解成多个短数据报片。在终点要把收到的各个短数据报片装配成原来的 TCP 报文段。当传输出错时还要进行重传。这些也都会使开销增大。MSS 应尽可能大些，只要在 IP 层传输时不需要再分片就行。
- ◆由于 IP 数据报所经历的路径是动态变化的，因此在这条路径上确定的不需要分片的 MSS，如果改走另一条路径就可能需要进行分片。
- ◆因此最佳的 MSS 是很难确定的。

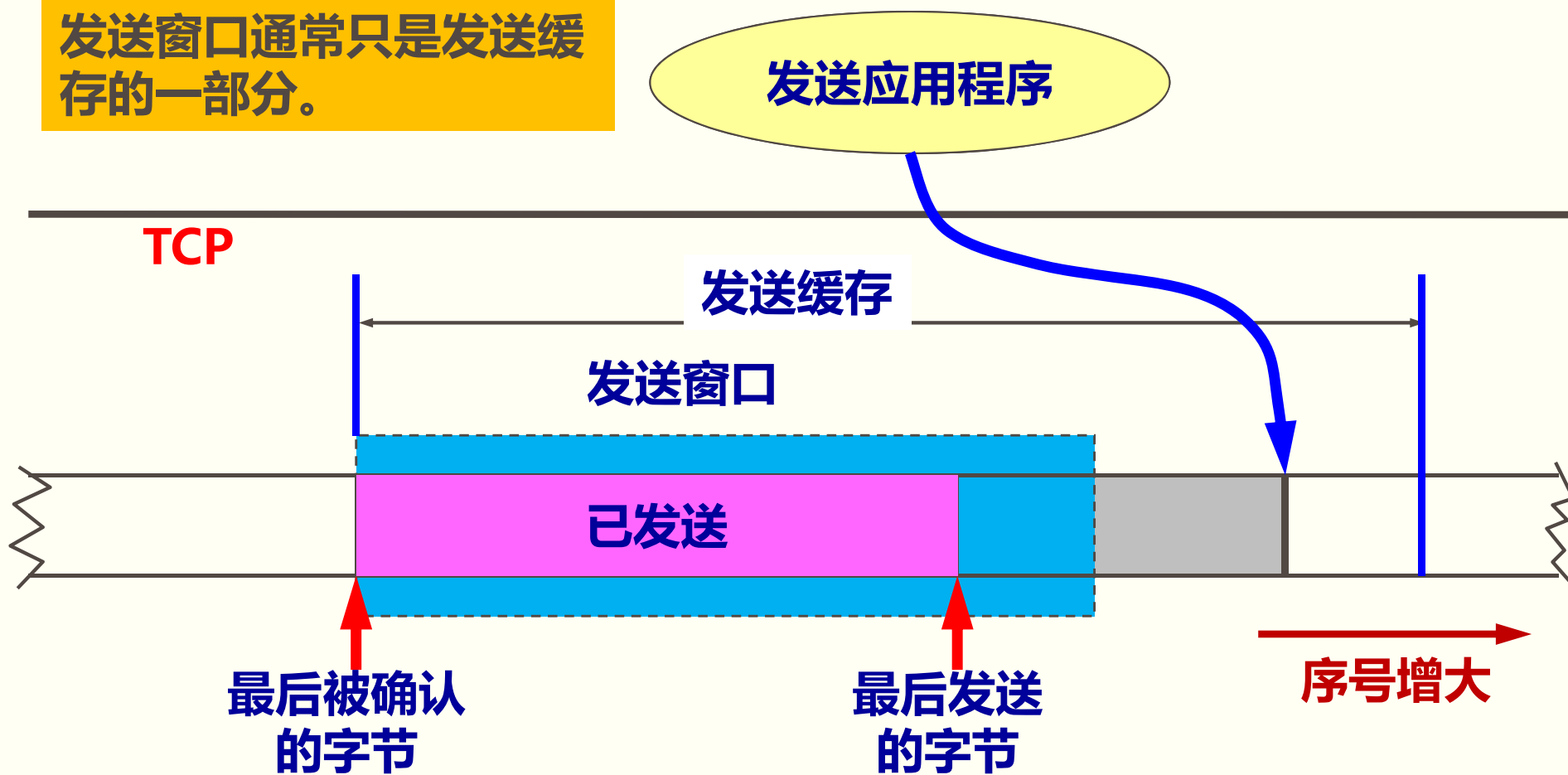


5.6 TCP 可靠传输的实现

发送缓存

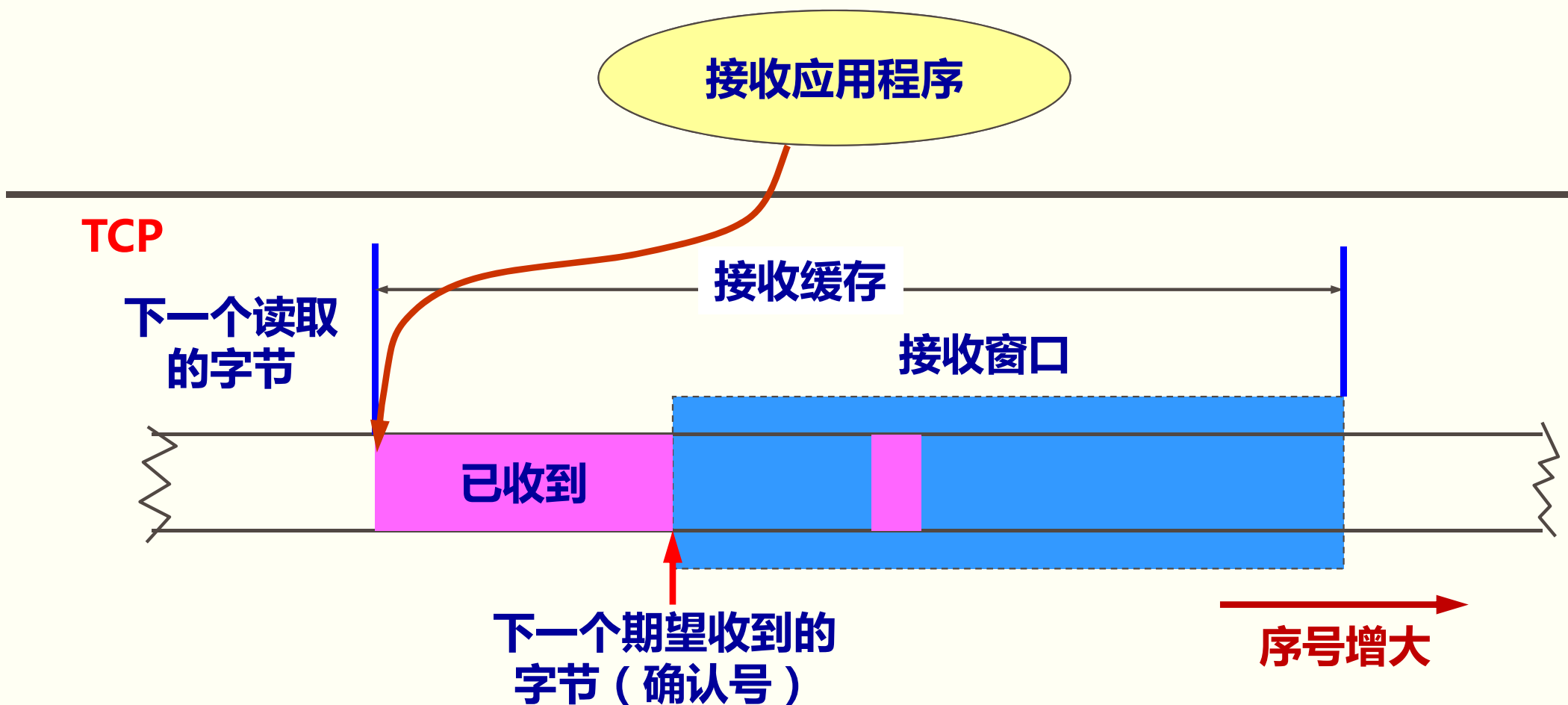
发送方的应用进程把字节流写入 TCP 的发送缓存。

发送窗口通常只是发送缓存的一部分。



接收缓存

接收方的应用进程从 TCP 的接收缓存 中读取字节流。



发送缓存与接收缓存的作用

◆发送缓存用来暂时存放：

- 发送应用程序传送给发送方 TCP 准备发送的数据；
- TCP 已发送出但尚未收到确认的数据。

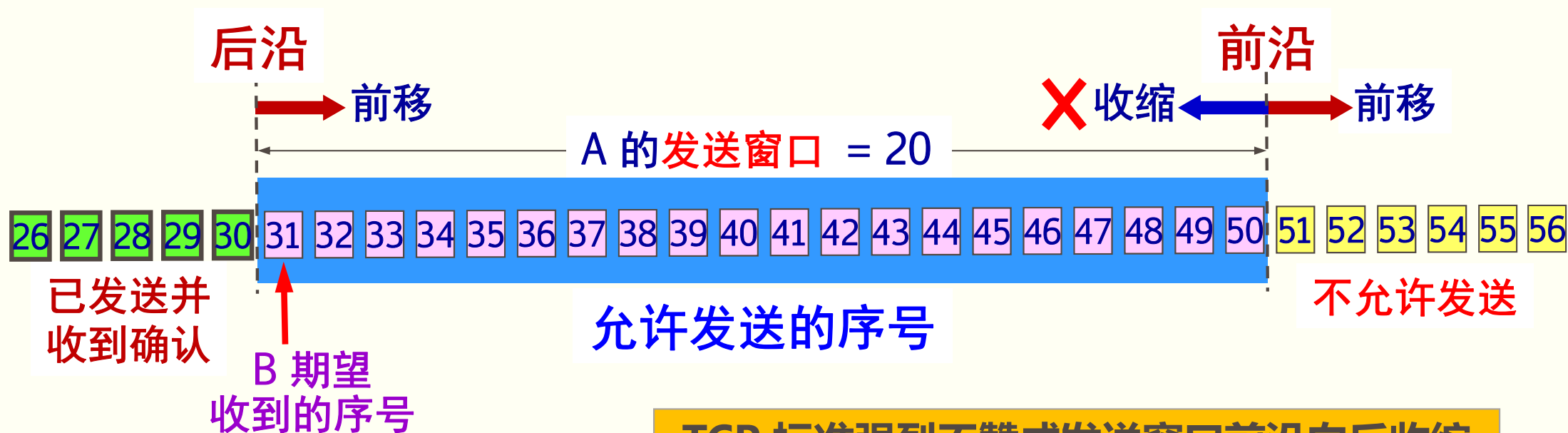
◆接收缓存用来暂时存放：

- 按序到达的、但尚未被接收应用程序读取的数据；
- 不按序到达的数据。

5.6.1 以字节为单位的滑动窗口**

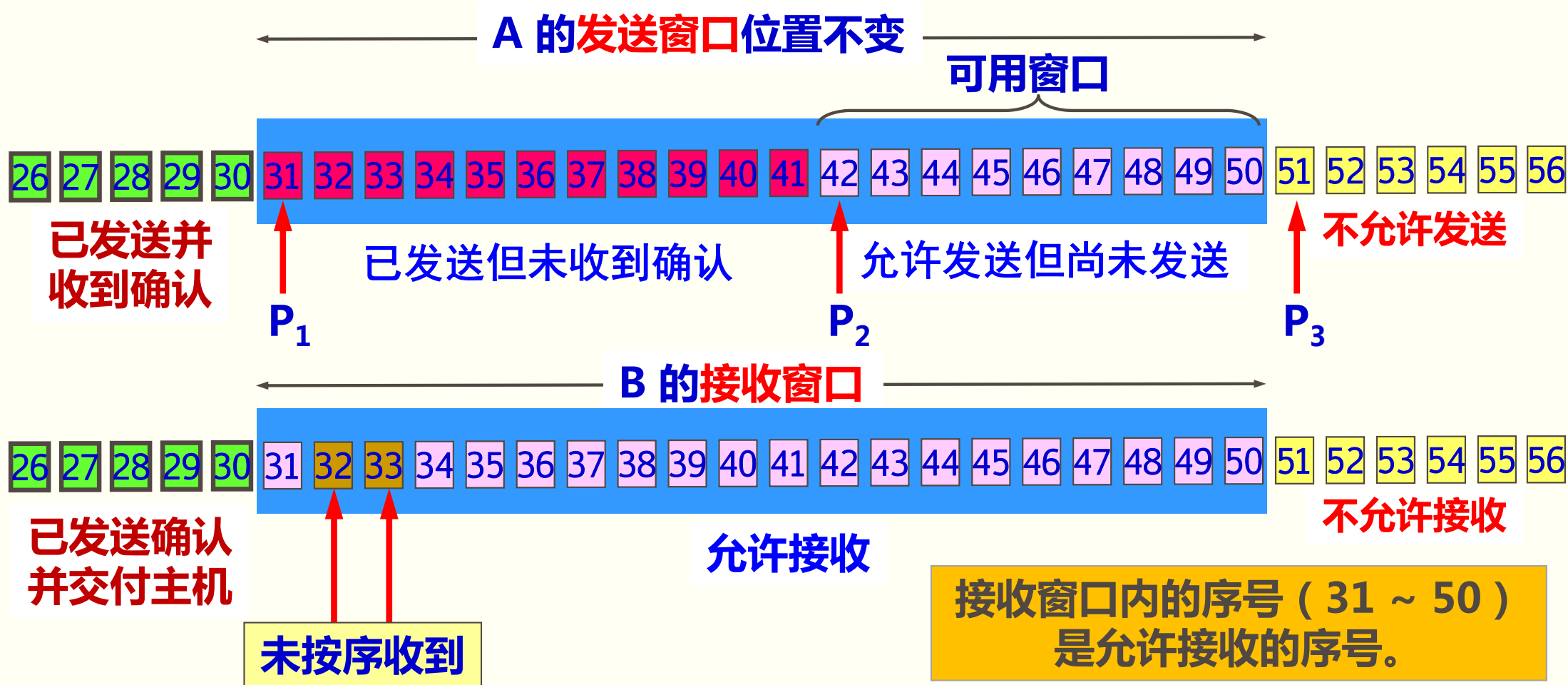
TCP 的滑动窗口是以字节为单位的。根据 B 给出的窗口值，A 构造出自己的发送窗口。发送窗口表示：在没有收到 B 的确认的情况下，A 可以连续把窗口内的数据都发送出去。发送窗口里面的序号表示允许发送的序号。

窗口越大，发送方就可以在收到对方确认之前连续发送更多的数据，因而可能获得更高的传输效率。



TCP 标准强烈不赞成发送窗口前沿向后收缩

A 发送了 11 个字节的数据

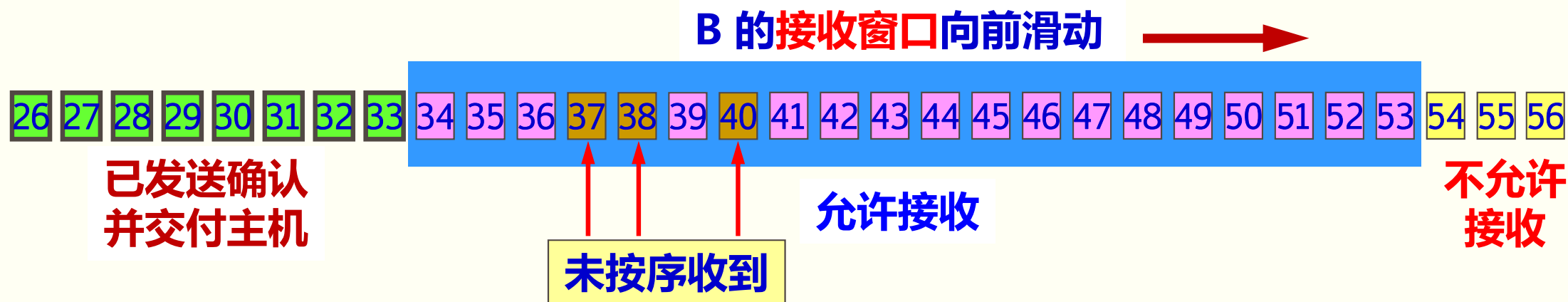
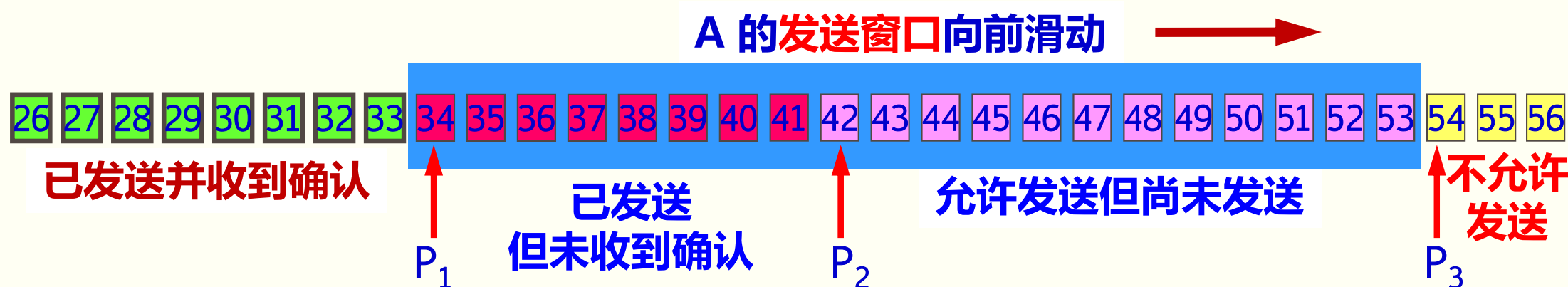


$P_3 - P_1 =$ A 的发送窗口 (又称为通知窗口)

$P_2 - P_1 =$ 已发送但尚未收到确认的字节数

$P_3 - P_2 =$ 允许发送但尚未发送的字节数 (又称为可用窗口)

A 收到新的确认号，发送窗口向前滑动



先存下，等待缺少的数据的到达

只有当未按序收到的数据到达接收窗口后沿时才会要求A重新发送

发送窗口内的序号都属于已发送但未被确认

A 的发送窗口内的序号都已用完，但还没有再收到确认，必须停止发送。

A 的发送窗口已满，有效窗口为零



同时A的发送窗口必须已经满了才会开始重新发送

需要强调

第一，A 的发送窗口并不总是和 B 的接收窗口一样大（因为有一定的时间滞后）。

第二，TCP 标准没有规定对不按序到达的数据应如何处理。通常是先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再按序交付上层的应用进程。

第三，TCP 要求接收方必须有累积确认的功能，这样可以减小传输开销。

◆接收方可以在合适的时候发送确认，也可以在自己有数据要发送时把确认信息顺便捎带上。注意两点：

第一，接收方不应过分推迟发送确认，否则会导致发送方不必要的重传，这反而浪费了网络的资源。。

第二，捎带确认实际上并不经常发生，因为大多数应用程序很少同时在两个方向上发送数据。

5.6.2 超时重传时间的选择

- ◆重传机制是 TCP 中最重要和最复杂的问题之一。
- ◆TCP 每发送一个报文段，就对这个报文段设置一次计时器。
- ◆只要计时器设置的重传时间到但还没有收到确认，就要重传这一报文段。
- ◆重传时间的选择是 TCP 最复杂的问题之一。

TCP 超时重传时间设置

- ◆如果把超时重传时间设置得太短，就会引起很多报文段的不必要的重传，使网络负荷增大。
- ◆但若把超时重传时间设置得过长，则又使网络的空闲时间增大，降低了传输效率。
- ◆TCP 采用了一种自适应算法，它记录一个报文段发出的时间，以及收到相应的确认的时间。这两个时间之差就是报文段的往返时间 RTT。

5.6.3 选择确认 SACK

◆**问题**：若收到的报文段无差错，只是未按序号，中间还缺少一些序号的数据，那么能否设法只传送缺少的数据而不重传已经正确到达接收方的数据？

◆答案是可行的。**选择确认 SACK**

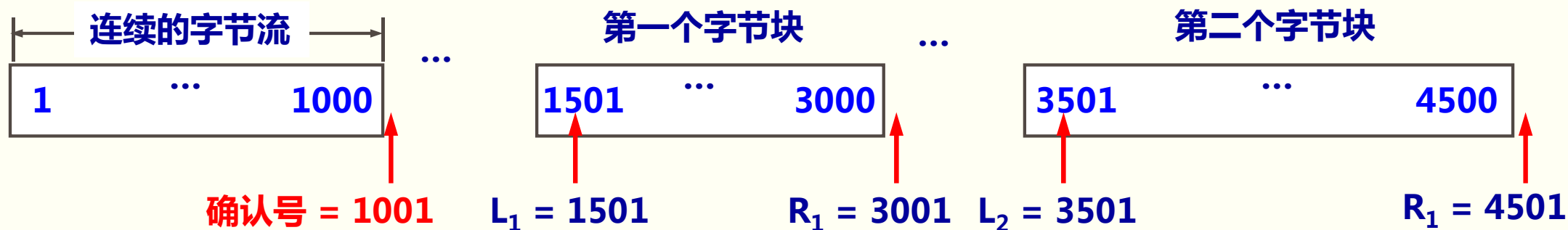
(Selective ACK) 就是一种可行的处理方法。

◆接收方收到了和前面的字节流不连续的两个字节块。

◆如果这些字节的序号都在接收窗口之内，那么接收方就先收下这些数据，但要把这些信息准确地告诉发送方，使发送方不要再重复发送这些已收到的数据。

接收到的字节流序号不连续

TCP 的接收方在接收对方发送过来的数据字节流的序号不连续，结果就形成了一些不连续的字节块。



和前后字节不连续的每一个字节块都有两个边界：边界和右边界。

- 第一个字节块的左边界 $L_1 = 1501$ ，但右边界 $R_1 = 3001$ 。左边界指出字节块的第一个字节的序号，但右边界减 1 才是字节块中的最后一个序号。
- 第二个字节块的左边界 $L_2 = 3501$ ，而右边界 $R_2 = 4501$ 。

RFC 2018 的规定

- ◆如果要使用选择确认，那么在建立 TCP 连接时，就要在 TCP 首部的选项中加上“允许 SACK”的选项，而双方必须都事先商定好。
- ◆如果使用选择确认，那么原来首部中的“确认号字段”的用法仍然不变。只是以后在 TCP 报文段的首部中都增加了 SACK 选项，以便报告收到的不连续的字节块的边界。
- ◆由于首部选项的长度最多只有 40 字节，而指明一个边界就要用掉 4 字节，因此在选项中最多只能指明 4 个字节块的边界信息。



5.9 TCP 的运输连接管理**

运输连接的三个阶段

- ◆ TCP 是面向连接的协议。
- ◆ 运输连接有三个阶段：
 - 连接建立
 - 数据传送
 - 连接释放
- ◆ 运输连接的管理就是使运输连接的建立和释放都能正常地进行。

TCP 连接建立过程中要解决的三个问题

- (1) 要使每一方能够确知对方的存在。
- (2) 要允许双方协商一些参数（如最大窗口值、是否使用窗口扩大选项和时间戳选项以及服务质量等）。
- (3) 能够对运输实体资源（如缓存大小、连接表中的项目等）进行分配。

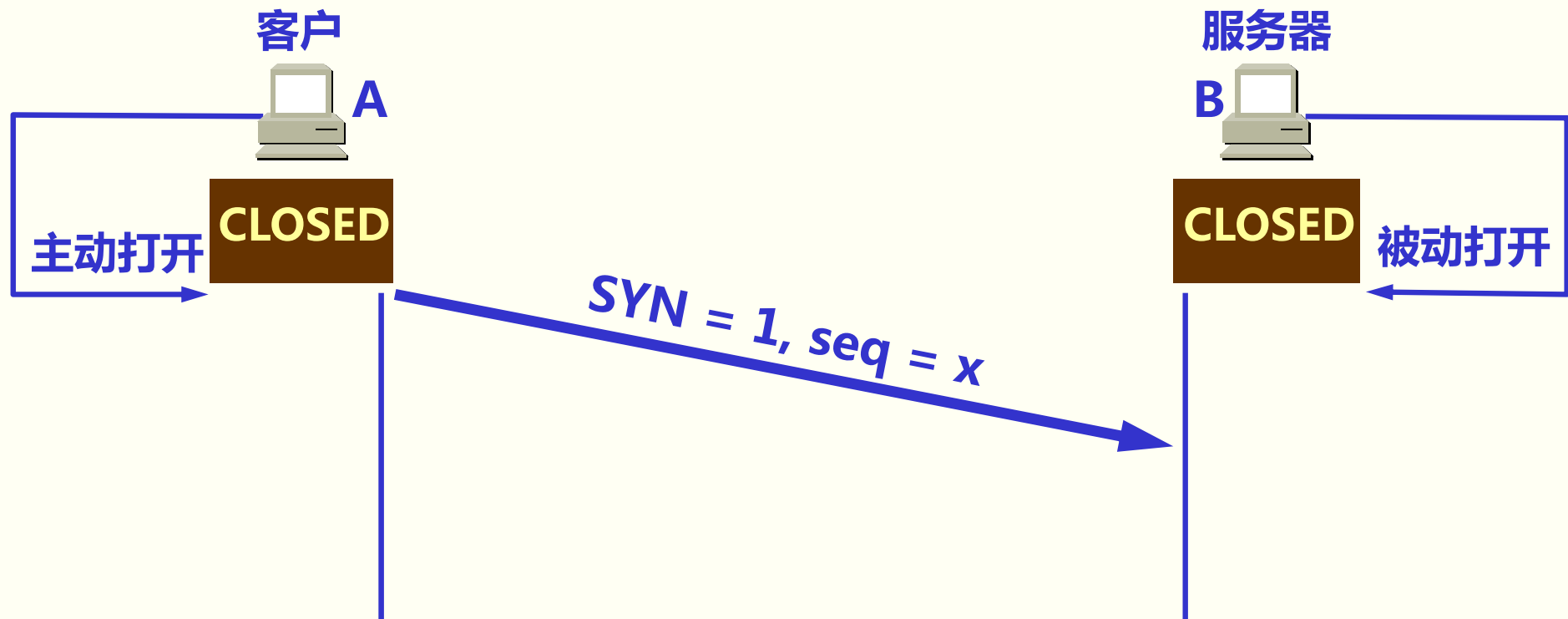
客户-服务器方式

- ◆TCP连接的建立采用客户服务器方式。
- ◆主动发起连接建立的应用进程叫做客户(client) ,
- ◆被动等待连接建立的应用进程叫做服务器(server)。

5.9.1 TCP 的连接建立**

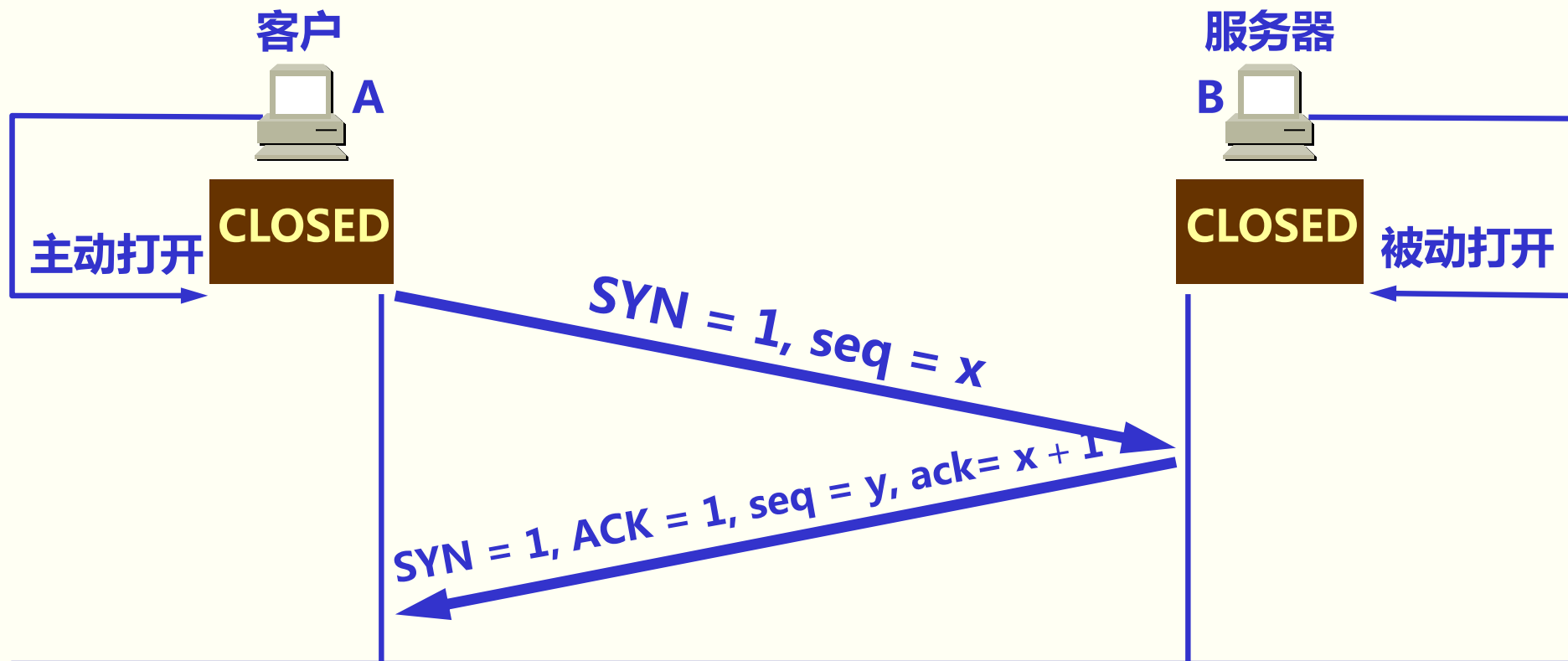
- ◆TCP 建立连接的过程叫做**握手**。
- ◆握手需要在客户和服务端之间交换三个 TCP 报文段。称之为**三报文握手**。
- ◆采用**三报文握手**主要是为了防止已失效的连接请求报文段突然又传**送**到了，因而产生错误。

TCP 的连接建立：采用三报文握手



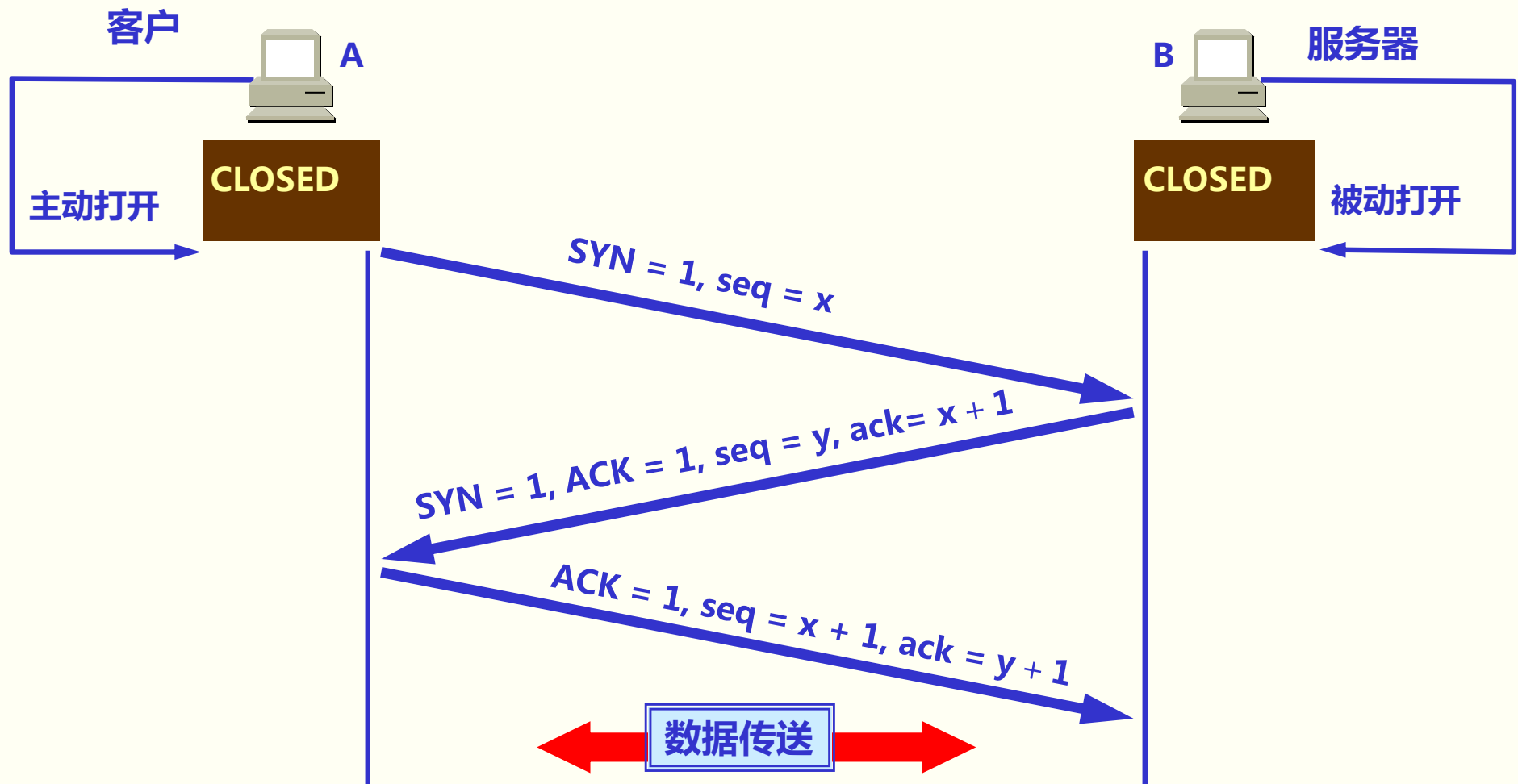
A 的 TCP 向 B 发出连接请求报文段，其首部中的同步位 $SYN = 1$ ，并选择序号 $seq = x$ ，表明传送数据时的第一个数据字节的序号是 x 。

TCP 的连接建立：采用三报文握手



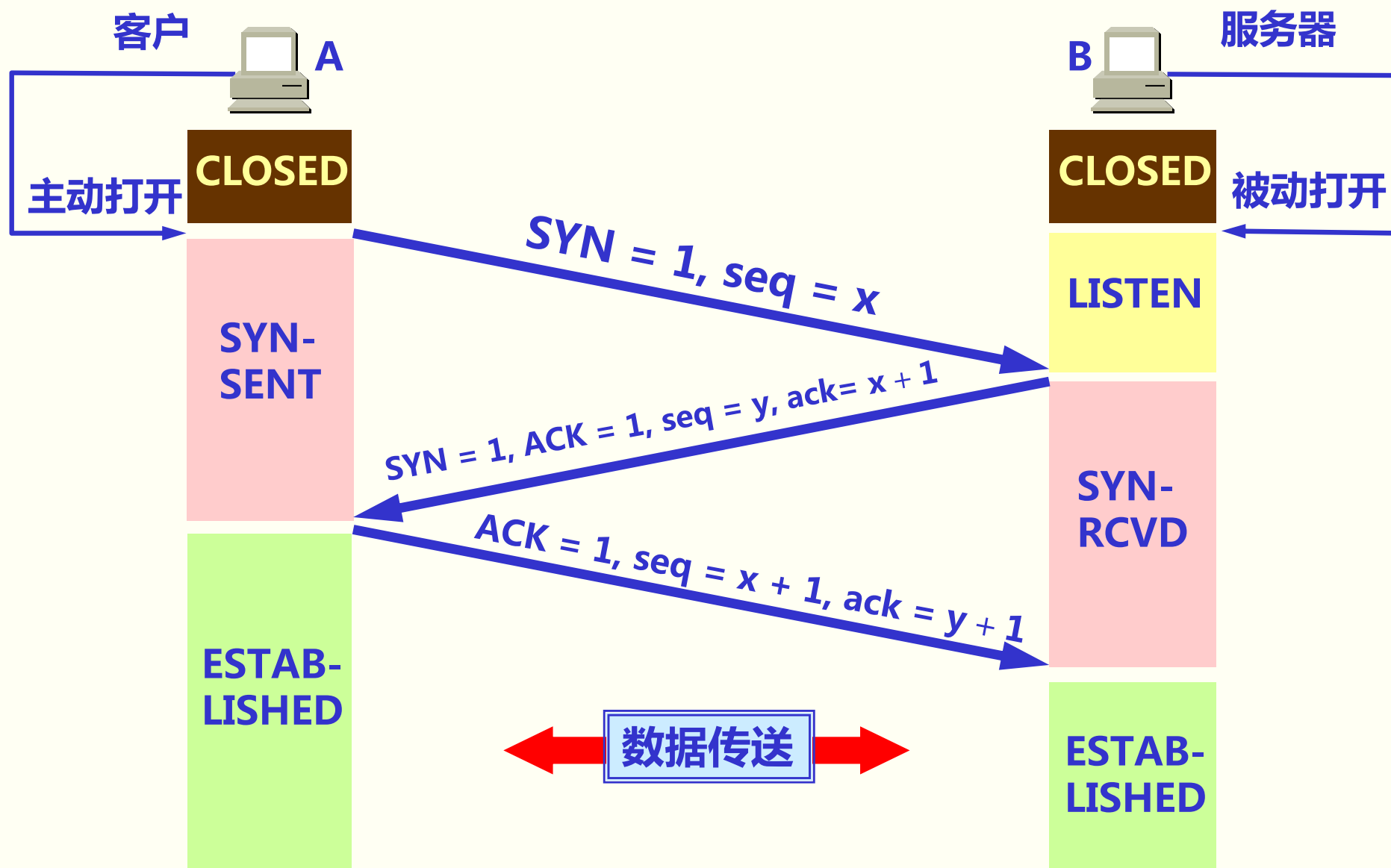
- B 的 TCP 收到连接请求报文段后，如同意，则发回确认。
- B 在确认报文段中应使 $SYN = 1$ ，使 $ACK = 1$ ，其确认号 $ack = x + 1$ ，自己选择的序号 $seq = y$ 。

TCP 的连接建立：采用三报文握手



- A 收到此报文段后向 B 给出确认，其 $ACK = 1$ ，确认号 $ack = y + 1$ 。
- A 的 TCP 通知上层应用进程，连接已经建立。
- B 的 TCP 收到主机 A 的确认后，也通知其上层应用进程：连接已经建立。

TCP 的连接建立：采用三报文握手

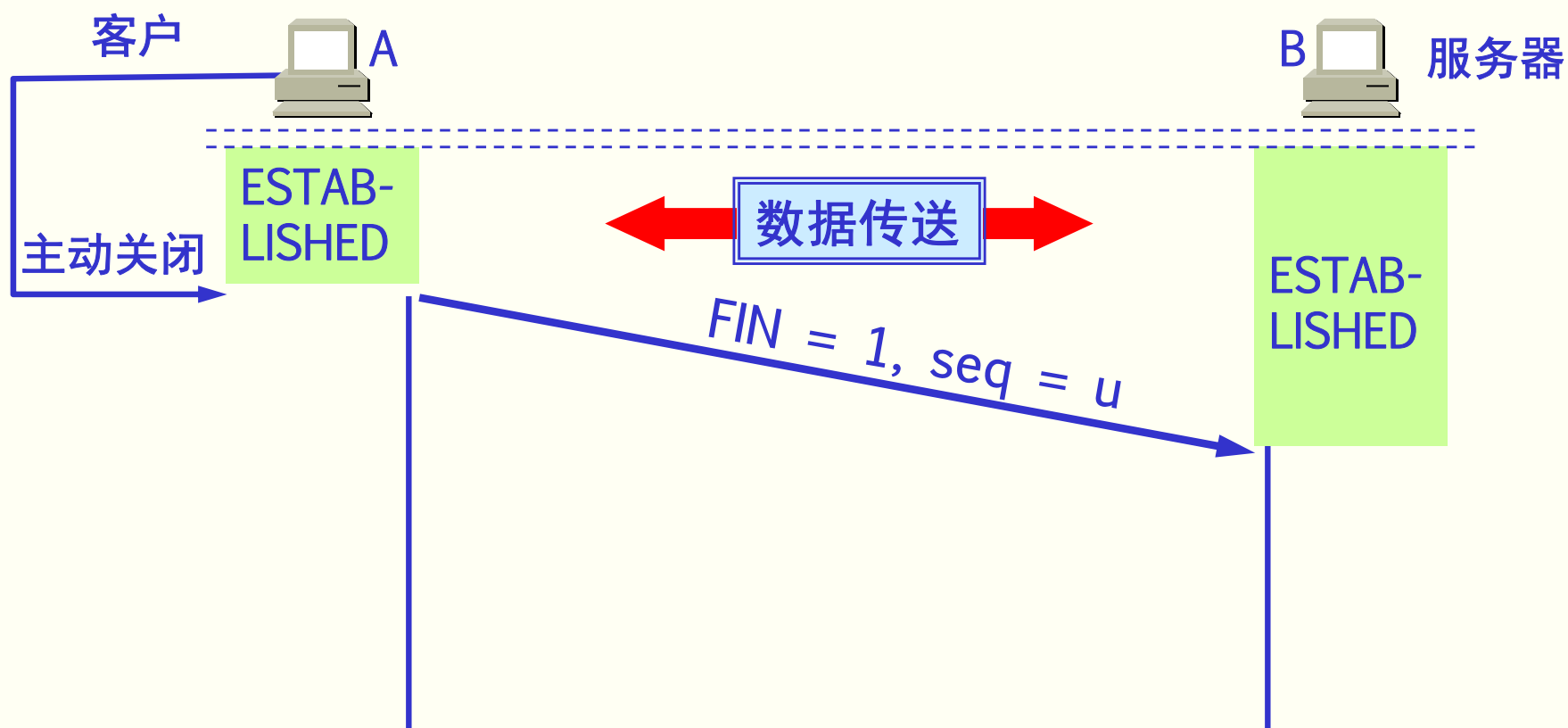


采用三报文握手建立 TCP 连接各状态

5.9.2 TCP 的连接释放**

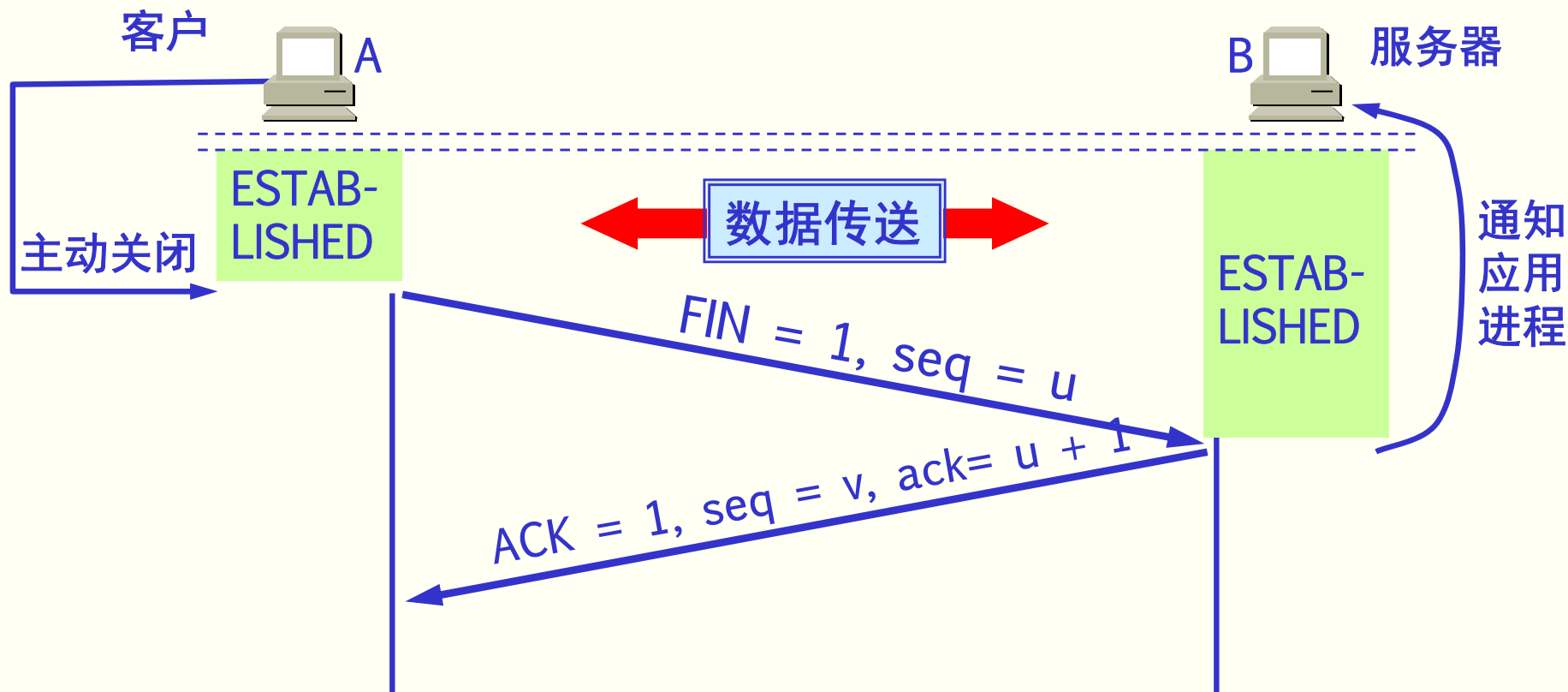
- ◆TCP 连接释放过程比较复杂。
- ◆数据传输结束后，通信的双方都可释放连接。
- ◆TCP 连接释放过程是四报文握手。

TCP 的连接释放：采用四报文握手



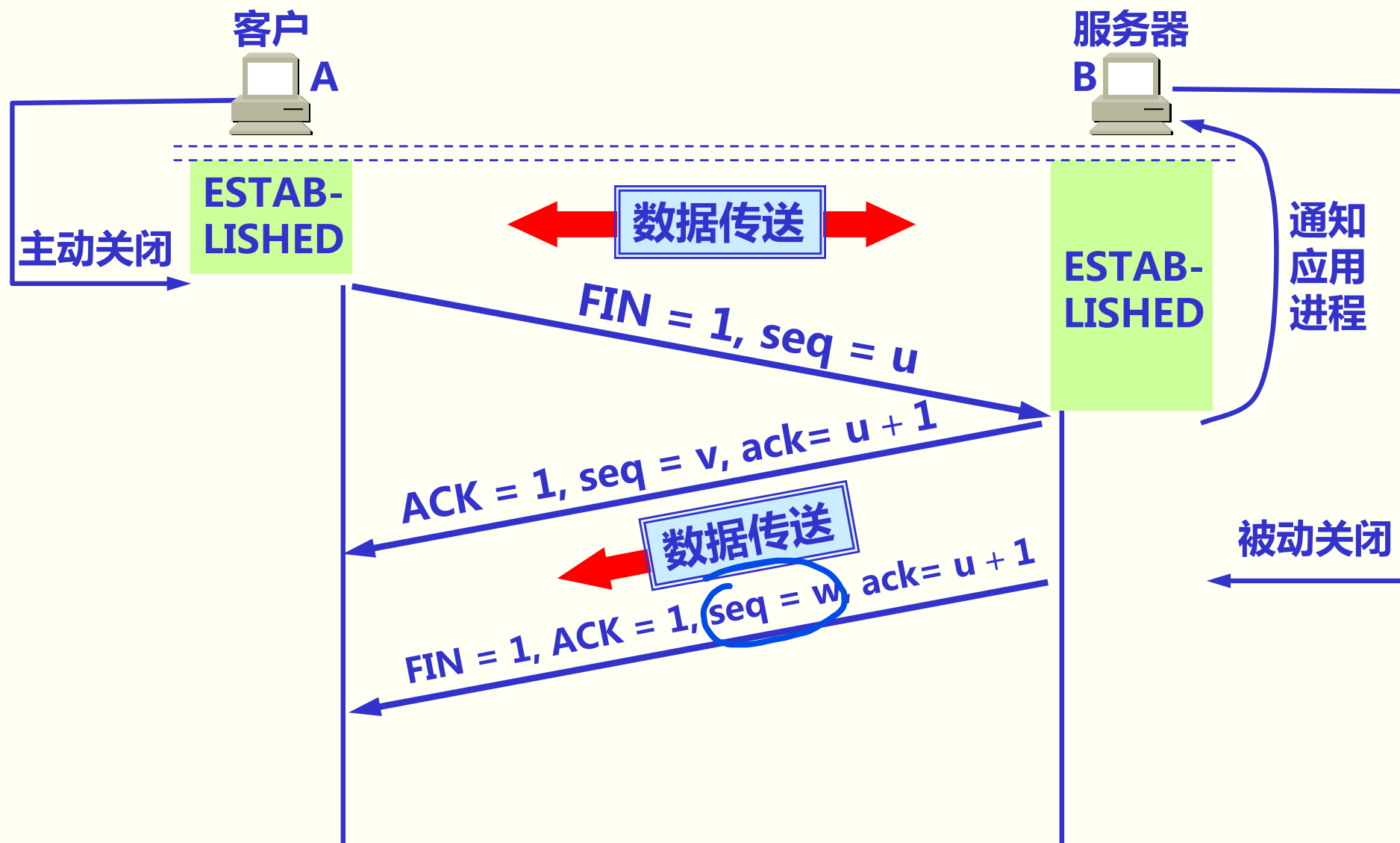
- 数据传输结束后，通信的双方都可释放连接。
- 现在 A 的应用进程先向其 TCP 发出连接释放报文段，并停止再发送数据，主动关闭 TCP 连接。
- A 把连接释放报文段首部的 $FIN = 1$ ，其序号 $seq = u$ ，等待 B 的确认。

TCP 的连接释放：采用四报文握手



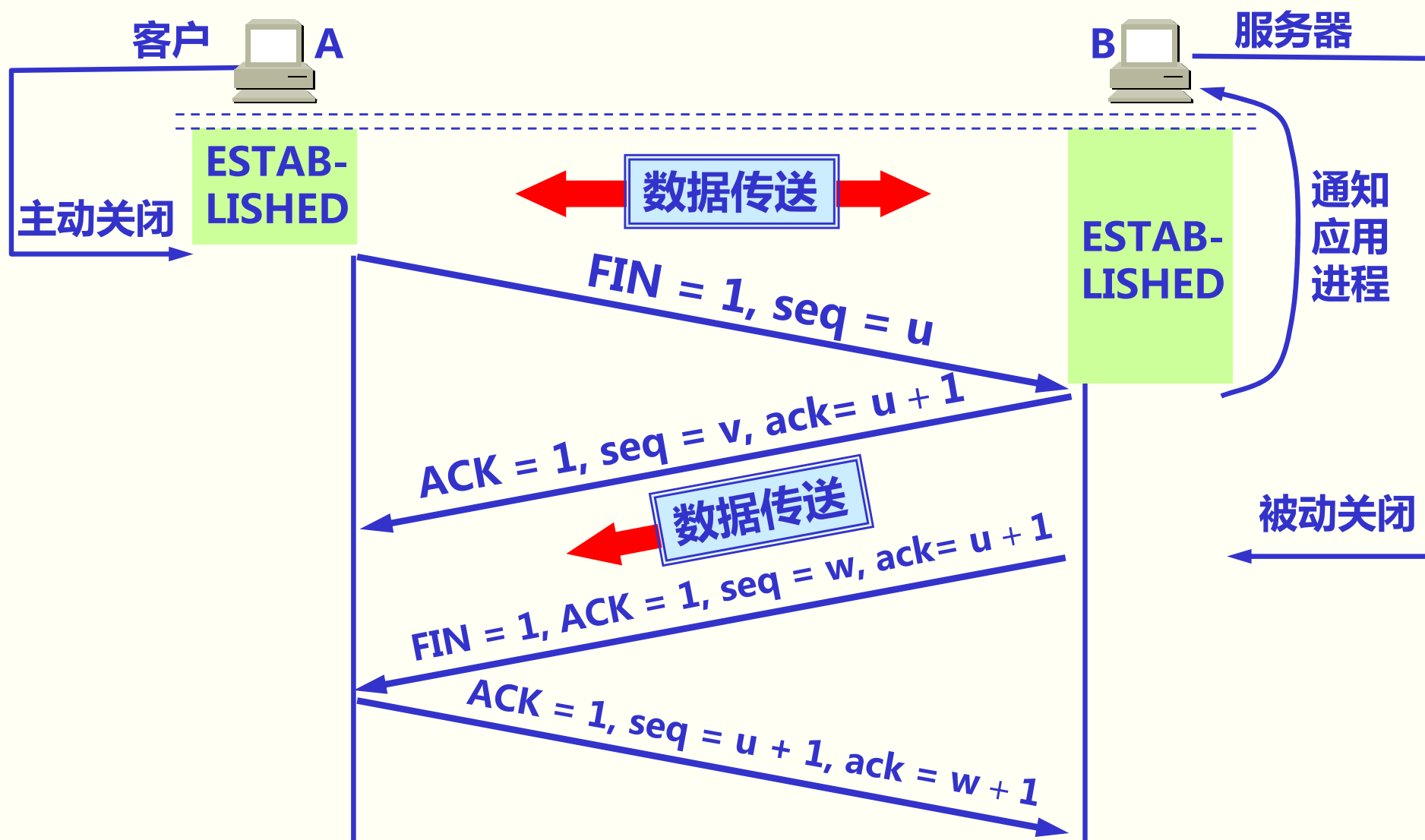
- B 发出确认，确认号 $ack = u + 1$ ，而这个报文段自己的序号 $seq = v$ 。
- TCP 服务器进程通知高层应用进程。
- 从 A 到 B 这个方向的连接就释放了，TCP 连接处于半关闭状态。B 若发送数据，A 仍要接收。

TCP 的连接释放：采用四报文握手



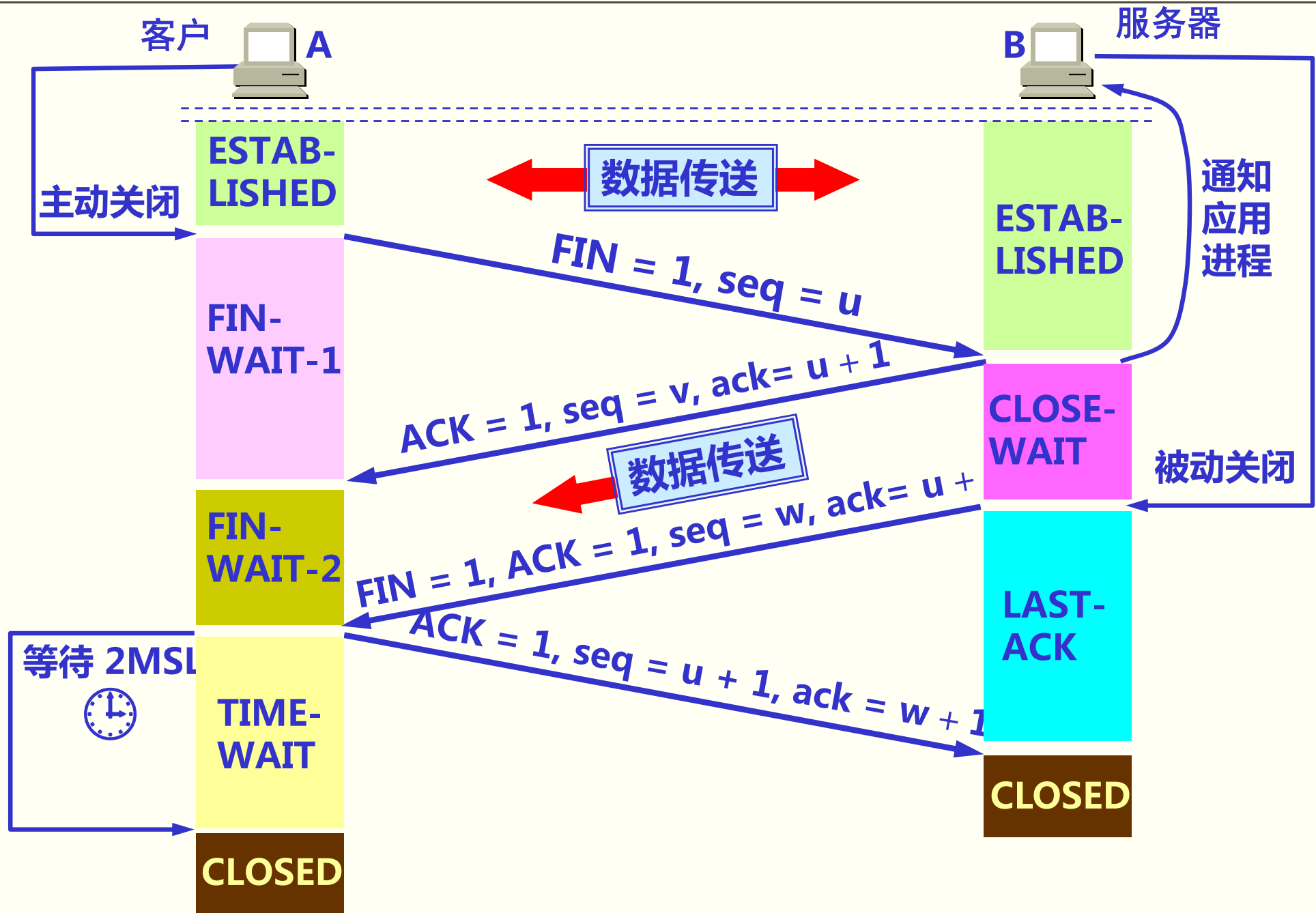
- 若 B 已经没有要向 A 发送的数据，其应用进程就通知 TCP 释放连接。

TCP 的连接释放：采用四报文握手



A 收到连接释放报文段后，必须发出确认。在确认报文段中 $ACK = 1$ ，确认号 $ack = w + 1$ ，自己的序号 $seq = u + 1$ 。

TCP 连接必须经过时间 2MSL 后才真正释放掉。



A 必须等待 2MSL 的时间

第一，为了保证 A 发送的最后一个 ACK 报文段能够到达 B。

第二，防止 “已失效的连接请求报文段” 出现在本连接中。A 在发送完最后一个 ACK 报文段后，再经过时间 2MSL，就可以使本连接持续的时间内所产生的所有报文段，都从网络中消失。这样就可以使下一个新的连接中不会出现这种旧的连接请求报文段。

5.9.3 TCP 的有限状态机

- ◆TCP 的有限状态机可以更清晰地看出 TCP 连接的各种状态之间的关系。
- ◆TCP 有限状态机的图中每一个方框都是 TCP 可能具有的状态。
- ◆每个方框中的大写英文字符串是 TCP 标准所使用的 TCP 连接状态名。
- ◆状态之间的箭头表示可能发生的状态变迁。

5.9.3 TCP 的有限状态机

- ◆箭头旁边的字，表明引起这种变迁的原因，或表明发生状态变迁后又出现什么动作。
- ◆图中有三种不同的箭头。
 - 粗实线箭头表示对客户进程的正常变迁。
 - 粗虚线箭头表示对服务器进程的正常变迁。
 - 细线箭头表示异常变迁。

TCP 的有限状态机

